



T1: Terminal Agent Reinforcement Learning for Long-Horizon Tasks

Junyao Yang^{1,2,*,†}, Yucheng Shi^{1,*}, Zhongzhi Li^{1,3}, Ruhan Wang^{1,4}, Zongxia Li^{1,5}, Haitao Mi¹, Leowei Liang¹

¹Tencent Hy Foundation Model Frontier ²National University of Singapore ³University of Georgia

⁴Indiana University ⁵University of Maryland, College Park

*Equal Contribution, †Corresponding Author

junyao yang@u.nus.edu, tberiusyang@global.tencent.com

Agent usage is shifting toward long-horizon tasks such as coding and scientific discovery, among which terminal tasks are especially important. We introduce **T1**, a Mixture-of-Experts model of 122B total trained with reinforcement learning, operating a real shell in a cloud sandbox for up to **300+ tool-call turns per task**, rewarded by executing each task’s own verifier. We provide a comprehensive recipe: First, an aggressively warm-started to stabilize actor-critic training, with a **dense process reward** scoring trajectories by the absolute number of passing verifiers. Second, stable optimization through **TITO** construction, training on the exact sampled token identifiers with drift repair at turn boundaries, and **rollout routing replay**, recording the sampler’s per-token expert choices at every MoE layer and replaying them during training. Third, **fully out-of-distribution training corpus**: isolated seeds and synthesized tasks disjoint from Terminal-Bench 2.1 ensures gains reflect genuine capability transfer over benchmark overfitting. Together, TITO and R³ cut the training-to-inference log-probability difference from 0.021 to 0.013, with exactly aligned **zero token drift** in the loss region. On Terminal-Bench 2.1, our post-train pipeline raises initial base model from 43.8% to **T1** with **64.0% resolved**. On Long-Horizon Terminal Bench, **T1** reaches **27.9%** and surpasses GPT-5.4 and GLM-5.1.

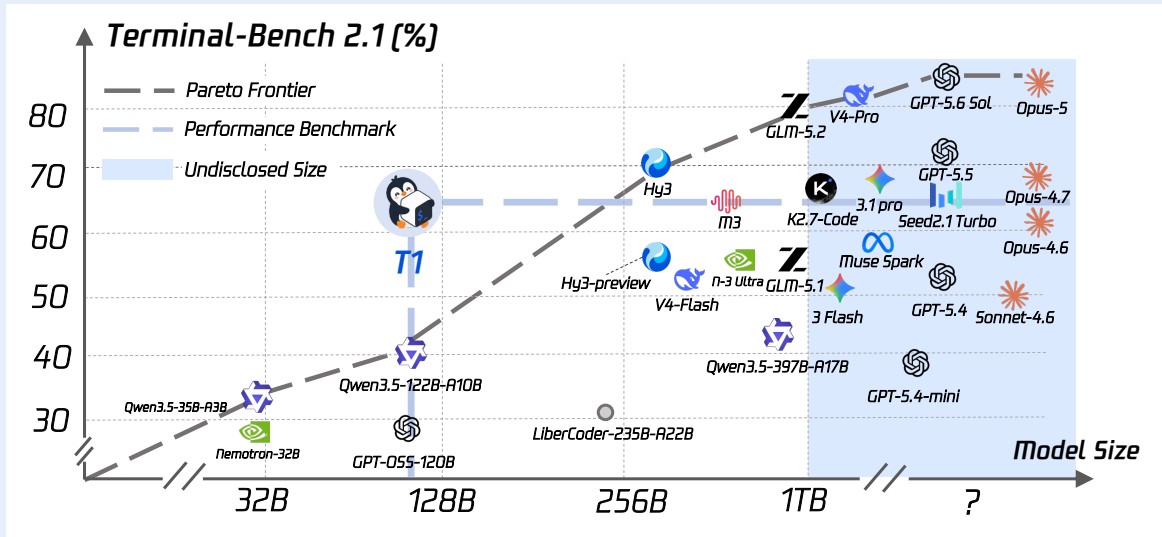


Figure 1 Performance vs. model size under the same agent harness (Terminal-Bench 2.1 resolved rate). At 122B, **T1** achieves 64.0%, outperforming GPT-5.4 (54.8%), DeepSeek-V4-Flash (56.9%), and Claude Opus 4.6 (63.8%). It closely approaches Claude Opus 4.7 (66.1%), and is the best model in its size band.

Project Page

HuggingFace

Date: September 10, 2026

Contents

1	Introduction	4
2	Training framework	5
2.1	Recursive Synthesis Terminal Training Tasks	5
2.2	Stable MoE Reinforcement Learning	6
2.3	Dense Verification Reward Design	6
2.4	Performance on Long-Horizon and Challenge Terminal Tasks	6
3	Terminal Dataset	6
3.1	Overview	6
3.2	Dataset Construction	7
4	Stabilizing MoE RL training	8
4.1	Training-Inference Mismatch	8
4.2	TITO: token-in, token-out	9
4.3	R ³ : rollout routing replay	10
4.4	Measuring Training Stability of T1	11
4.5	Critic-side stability	12
5	Dense Verification Reward Design	13
5.1	From sparse outcomes to dense verification signals	13
5.2	Per-assertion verification and the test-count reward	14
5.3	Credit assignment	15
5.4	Preventing reward hacking	16
6	Results	16
6.1	Benchmark and Harness	16
6.2	Main results	17
6.3	Training dynamics	19
6.4	Case study: detailed evaluation across benchmarks	20
7	Infrastructure	22
7.1	A capacity model for co-resident actor-critic pairs	22
7.2	Context parallelism for the recurrent operator	23
7.3	Liveness under hour-scale steps	23
7.4	Environment concurrency and the straggler tail	24
7.5	The publication barrier and rollout/training balance	24
8	Related Work	25
9	Lessons learned and what did not work	25
9.1	Critic calibration does not guarantee policy improvement	26
9.2	Context management determines what the agent can learn	26
9.3	Rollout throughput changes the training distribution	26
9.4	Why PPO rather than a critic-free group baseline	27
10	Limitations and future work	27
A	Notation	31
B	Asynchrony, version skew, and routing alignment	33
B.1	The one-step-ahead loop	33

B.2	Three distinct gaps, one measured quantity	33
B.3	Routing alignment across a version boundary	34
B.4	Version bookkeeping in the critic path	34
C	Case Study	35

1 Introduction

Large language models have pushed agentic AI from single-turn code completion and conversational assistance toward the far more demanding domain of autonomous long-horizon execution [14]. This marks a critical frontier: a model must no longer merely produce text satisfying a rubric, but issue actions whose consequences persist in a stateful environment and withstand verification by execution rather than preference. Among such environments the **Linux terminal** is the sharpest and most unforgiving test, since it binds abstract planning to irreversible side effects and forms the substrate of nearly all modern software engineering.

Mastering the terminal demands more than command syntax: environment comprehension, task decomposition, and precise recovery from partial failure, sustained over horizons far exceeding ordinary reasoning benchmarks. These skills are most rigorously tested by **long-horizon terminal benchmarks** such as Long-Horizon Terminal-Bench [9], Terminal-Bench Hard [8] and Terminal-Bench [14], where one task may require bisecting hundreds of commits, repairing a defect, rebuilding to a named target and proving the repair, adjudicated by the task’s own held-out verifier. We view reinforcement learning on executed outcomes as the critical path toward *autonomous software agency*: before models can operate production systems, reward derived from real execution rather than a learned preference model must be shown to optimize stably at frontier scale. Terminal performance is thus not the end goal, but a step toward agents acting on consequential infrastructure.

In this work we introduce **T1**, obtained by post-training Qwen3.5-122B-A10B [28], through reinforcement learning [19, 23] on terminal tasks. Our design confronts the two difficulties that dominate this regime:

- **Training-inference consistency for sparse models.** Expert weights account for 116.0B of 121.4B parameters, and each token engages 8 of 256 experts per layer through a discrete router. Minor numeric differences between inference and training flip these selections, so gradient may reach different parameters from those that generated the behaviour, while multi-turn harnesses perturb the token sequence at every turn boundary. We treat these as orthogonal axes, resolved separately by **TITO** for tokens and **R³** for experts (Sections 4.2 and 4.3), cutting the measured log-probability gap from 0.021 to 0.013 with **zero drift** for training.
- **Dense reward from execution.** A rollout batch costs hundreds of sandbox-hours, yet a binary outcome yields one bit per trajectory; our first binary-reward campaign never exceeded its supervised baseline. We instead score by the **absolute number of passing assertions** on a fixed global scale, feeding a warm-started critic trained at 30× the actor learning rate (Sections 5 and 4).

We evaluate on Terminal-Bench 2.1, 89 held-out tasks scored by execution. From a supervised checkpoint at 49.4%, three epochs of PPO on the quality-filtered T1-15k reach **64.0% resolved**, a **28.5% relative** gain from RL alone. Under an identical harness this places **T1** above GPT-5.4 at 54.8% and DeepSeek-V4-Flash at 56.9%, approaching Claude Opus 4.7 at 66.1%, the strongest model in its size band (Section 6). Gains concentrate where terminal agency is tested: **100.0** on debugging and **88.9** on system administration, both surpassing a stronger general-purpose model.

Our training corpus is moreover fully **out-of-distribution** with respect to the evaluation: isolated seeds and synthesized tasks disjoint from Terminal-Bench 2.1 ensures gains reflect genuine capability transfer over benchmark overfitting, so gains reflect transfer rather than benchmark fitting (Section 3).

Contributions. Our contributions are threefold:

- We introduce **T1**, a 122B MoE terminal agent trained purely by reinforcement learning on executed outcomes, up to **300+ tool-call turns** per task.
- We present a **stabilization stack** for large-scale sparse agentic RL, combining TITO, R³ and a scheduled critic, with the infrastructure keeping a co-resident 122B actor–critic pair alive for days (Section 7).
- We contribute a **dense execution reward** with its measured behaviour and two shaping variants that failed, alongside a candid record of failures (Section 9), which we found as instructive as the successes.

Together, these advances mark a significant step toward language models that act reliably in consequential environments rather than merely describing how to do so.

2 Training framework

Figure 2 maps the system from left to right across three main components. A training backend and inference replicas run concurrently on disjoint accelerators under slime framework [23], pipelining step t training with step $t + 1$ generation to hide latency. Our terminal-agent integration attaches additively via public extension points. The following sections detail each ingredient: **Tasks** providing verifiable inputs, the **Training Framework** managing asynchronous rollouts and updates, and the **Sandbox** executing multi-turn commands for reward collection.

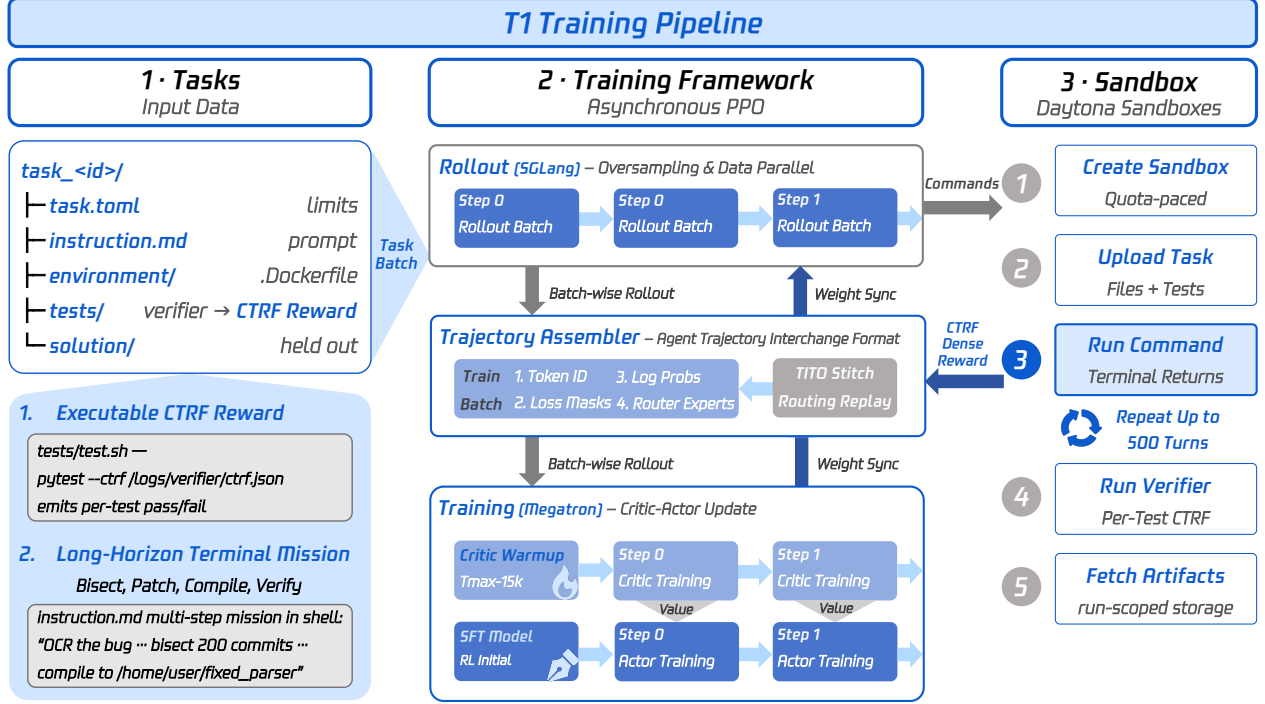


Figure 2 The **T1** training pipeline. **Left:** each task is self-contained, comprising metadata, a long-horizon instruction, an environment image with resource limits, a held-out verifier and a reference solution, and its verifier reports each assertion individually, which makes the reward executable rather than modelled. **Middle:** the inference replicas serve the behaviour policy under oversampling; the trajectory assembler normalizes interaction logs into TITO-stitched training samples carrying routing records and the dense reward; the trainer updates the critic and then the actor before synchronizing weights back. **Right:** sandboxes are created, loaded, driven turn by turn, verified and reclaimed. Rollout and training run concurrently on disjoint devices.

2.1 Recursive Synthesis Terminal Training Tasks

No reinforcement signal can be richer than what its verifier can measure, which makes task construction a design decision rather than a preprocessing step; Section 3 presents the resulting pools. Based on RST [8], each task is self-contained: resource limits, a long-horizon instruction such as bisecting hundreds of commits to locate a defect, patching it and proving the fix, a specification from which an isolated container is built, a held-out verifier, and a reference solution the agent never sees. The verifier is the load-bearing part, because it reports the outcome of every assertion separately rather than a single pass or fail, which is what makes the reward executed rather than modeled. **T1 utilized a selected proportion of high-quality 15K from RST as training set** based on multi-dimensions of the tasks from verifier, solution, instruction and value, following the audit developed in Section 3.2. A per-epoch seeded permutation then keeps the reward distribution from drifting with task quality.

2.2 Stable MoE Reinforcement Learning

Between the trajectory a sandbox produces and the gradient the trainer applies lie dozens of turns, two independent execution stacks and a discrete router, and each of them can silently attribute the update to a policy that never generated the data. Inference replicas serve the behaviour policy to many concurrent trials, and the scheduler oversamples, admitting the first to complete and cancelling the straggler tail to bound step time against heavy-tailed completions. Each trial drives its own sandbox, issuing commands and reading their terminal output turn after turn until the task completes or its limits are reached, whereupon the verifier runs and the sandbox is reclaimed, as Section 7 describes. The agent exchanges token identifiers rather than text, receiving back their log-probabilities and the expert routing chosen at every layer. An assembler normalizes multi-turn logs into training samples in which sampled tokens carry loss while tool output enters as masked context, and all four per-token streams are transformed by the same offsets at every stage, which is the pipeline’s central invariant. Section 4 shows how three mechanisms build on it to close the gap above: **Token-In-Token-Out**, **Routing Replay**, and **Infrastructure for Long-Horizon MoE Training**. The trainer updates the critic before the actor, since its pre-update values anchor the advantage estimator and because both networks time-multiplex the same devices. Weights are published once per step with generation quiesced first, so no request spans two versions and the lag from pipelining is off-policy-ness of exactly one step.

2.3 Dense Verification Reward Design

A single rollout batch costs hundreds of sandbox-hours, and a binary outcome repays that expense with one bit per trajectory; Section 5 spends the verifier’s full resolution instead. Each trial’s per-assertion outcome becomes a **Dense Process Reward**, scored by the absolute number of assertions satisfied on a scale fixed once for the whole run, so that harder tasks carry proportionally more signal and the critic sees a target comparable from step to step. The scalar enters at the final response token and the critic distributes credit across the horizon; with a single sample per task there is no group statistic to normalize against, leaving the critic as the only baseline. Because such a reward can in principle be farmed rather than earned, we filter the pool for verifiers too weak to validate their own goal and monitor trajectory growth throughout training.

2.4 Performance on Long-Horizon and Challenge Terminal Tasks

To determine the overall performance of **T1**, we conduct comprehensive evaluation between multiple frontier models and baseline models **Terminal-Bench 2.1** [14], **Long-Horizon Terminal-Bench** [9] and **Terminal-Bench Hard** [8] in Section 6. Three epochs of PPO lift the supervised checkpoint from 49.4% to **64.0%** resolved on Terminal-Bench 2.1, placing **T1** above GPT-5.4 and DeepSeek-V4-Flash with an order of magnitude fewer active parameters, and the gains hold where they matter most. On Long-Horizon Terminal Bench, whose tasks stress far longer horizons and are therefore the closer proxy for what this recipe optimizes, **T1** reaches **27.9%** and matches the performance of Gemini-3.1-Pro; on the harder Terminal-Bench Hard subset it reaches **38.0%**, ahead of DeepSeek-V4-Pro and well above both the supervised and the base checkpoint.

3 Terminal Dataset

3.1 Overview

A task is self-contained: per-trial limits, a long-horizon terminal instruction, an environment specification, and a held-out verifier with a reference solution the agent never sees. Three pools appear in our campaigns.

- **TMax-15k**: 14,601 tasks converted from the public corpus into terminal-bench layout. Its verifiers emit only a binary outcome with no per-assertion record, so only binary rewards are possible here.
- **RST-38k**: 37,484 synthesized tasks generated via RST [8], which iteratively extends seed solutions, realigns verifiers and instructions, and sandboxes each task before recursive seeding.
- **T1-15k**: 15,000 tasks selected from the synthesis rounds by the audit of Section 3.2. Their verifiers report per-assertion outcomes, and a pre-flight confirmed such records in 93% of sampled tasks. The

pool is materialized in quality-rank order, so per-epoch shuffling is mandatory (Section 5.4).



Figure 3 Category composition of T1-15k. All 15,000 tasks are counted once across 17 merged categories (from 47 raw labels); angle encodes share exactly, while radius is a rank-based power scale chosen to keep small slices visible and is therefore *not* proportional to share. The pool is concentrated in command-line engineering work: the top five categories account for 67.9% of all tasks.

Because T1-15k carries the dense-reward runs, its composition is worth stating. Figure 3 gives the breakdown: scripting and automation at 17.9%, software development at 16.5%, system administration at 13.8%, environment and package setup at 10.4% and version control at 9.3% together make up two-thirds of the pool, while data science at 3.7%, debugging at 1.3% and performance work at 1.0% are thin. This skew predicts where residual failures land, as Section 6.4 confirms.

3.2 Dataset Construction

Both synthesized pools are produced by recursive task synthesis [8], which grows a curriculum rather than sampling tasks independently. Each round takes accepted tasks from the previous round as seeds under caps on parent lineage, category and rewrite family; for each seed it selects a feasible rewrite operator, extends the reference solution with additional executable steps, then aligns the environment, verifier and public instruction to that longer workflow. Candidates are validated in a fresh sandbox where the reference solution must genuinely pass the held-out verifier, with bounded repair for recoverable failures and discard otherwise. Because difficulty is added to the executable path before the instruction is rewritten, horizons lengthen without the task degenerating into a longer prompt over the same behaviour.

Synthesis yield alone does not make a pool a usable RL signal, so T1-15k is the subset of those rounds surviving an LLM audit. Figure 4 shows what that audit optimizes for. Alignment between the instruction and the verifier carries the largest weight at 20%, because a mismatch between the public instruction and what the held-out verifier enforces constitutes a hidden requirement and makes the task unusable: the agent is punished for failing a criterion it was never shown. The remaining weight splits between verifier quality, at 15% for fairness and 10% for coverage, and solution quality, at 15% for correctness and 10% for reasonableness, with task training value the only dimension exempt from the critical-minimum cutoff. Tasks are hard-rejected for hidden requirements, test leakage, solution shortcuts, or verifiers too weak to validate the goal. Selection

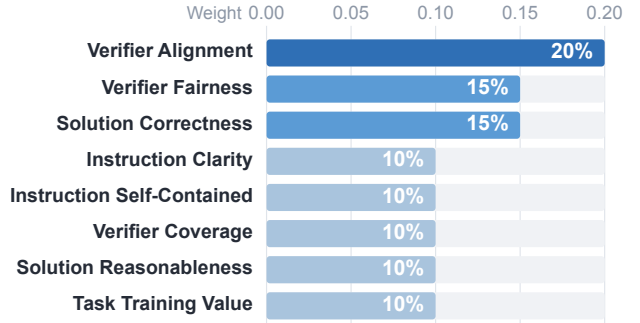


Figure 4 Figure 4: The eight audit dimensions and their weights, summing to 1.00. Aggregated by facet: Verifier 45%, Solution 25%, Instruction 20%, and Task Value 10%. Darker bars highlight the highest-weighted dimensions

ran in five stages over 15 rewrite rounds: aggregation with lineage; static pre-check with executable validation; a semantic pass yielding 5,902 accepted, 3,251 borderline and 5,847 rejected; instruction-only repair of 6,875 tasks; and a re-audit.

4 Stabilizing MoE RL training

4.1 Training-Inference Mismatch

Setting. Rollout and training are served by two different systems: generation runs on inference replicas built for throughput, with fused kernels, batched prefill and a paged key-value cache, while the update runs on a training backend built for exact gradients, with its own kernels, reduction orders and tensor layouts. The two agree on the parameters they hold and on little else. Between them sits the agent harness, which exchanges no tensors at all: it persists each assistant message as parsed text and re-renders the whole history through a chat template before every turn.

The loop is additionally one-step asynchronous, so generation for step $t+1$ overlaps the update at step t . Writing π_t for the policy with parameters θ_t , the batch consumed by update t was generated under π_{t-1} , and the per-token objective rests on the ratio

$$r_j(\theta) = \frac{\pi_t}{\pi_{t-1}}. \quad (1)$$

That much is legitimate and fully modelled off-policyness of exactly one update, since π_{t-1} is a policy we did hold and the clip bounds how far π_t may travel from it. What is *not* modelled is which system evaluates the denominator. We recompute it on the training side, so validity demands that the trainer reproduce at version $t-1$ what the sampler realized at that version. Marking evaluation by the sampler and by the trainer with superscripts r and t, the requirement $\pi_{t-1}^t \equiv \pi_{t-1}^r$ separates into two independent conditions.

Two fidelity conditions. Let T_j be the token identifier at position j of the assembled trajectory and $\mathbb{I}_j^\ell$ the *routing mask* at MoE layer ℓ , the set of experts the router selects for that position by a discrete top- k over E candidates,

$$\mathbb{I}_j^\ell = \text{TopK}_k(\mathbf{s}_j^\ell), \quad \mathbf{y}_j^\ell = \sum_{e \in \mathbb{I}_j^\ell} [\mathbf{s}_j^\ell]_e f_e(\mathbf{h}_j^\ell), \quad (2)$$

with $(L, k, E) = (48, 8, 256)$ for Qwen3.5-122B-A10B. Because $\mathbb{I}_j = (\mathbb{I}_j^1, \dots, \mathbb{I}_j^L)$ decides *which* experts act, it indexes a sub-network of the 95.5% of parameters held by experts, so the policy must be written $\pi_\theta(\cdot \mid T_{<j}, \mathbb{I}_j)$. For every position carrying loss, Equation 1 therefore compares two versions of one policy only if

$$\text{token fidelity} \quad T_j^t = T_j^r, \quad (3)$$

$$\text{routing fidelity} \quad \mathbb{I}_j^{t,\ell} = \mathbb{I}_j^{r,\ell} \quad \forall \ell \in [L]. \quad (4)$$

The harness breaks token fidelity: re-rendering the history returns turn i 's output as $\text{enc}(\text{dec}(\mathbf{a}_i))$, and that round trip is not the identity whenever parsing normalizes the message or the template re-tokenizes at a boundary, so the trainer conditions on a stream the sampler never produced. The two stacks break routing fidelity: they compute $\mathbf{s}_j^\ell$ by different kernels, and since TopK_k is discontinuous, a numeric difference far below any tolerance one would place on a logit suffices to exchange a selected expert for its runner-up, substituting one sub-network for another so that the ratio relates two different networks rather than two versions of one.

Why both must be enforced. The conditions are independent, so enforcing either leaves the other's failure mode intact. Dense models cannot violate routing fidelity at all and short-horizon tasks make token fidelity nearly automatic, whereas the model emitting tens of tool-calling turns violates both across roughly 10^4 loss-bearing positions, where per-position discrepancies accumulate along the trajectory instead of cancelling. Section 4.2 enforces token fidelity by having the trainer consume the identifiers the sampler emitted, repairing turn boundaries under a small auditable set of cases; Section 4.3 enforces routing fidelity by recording $\mathbb{I}_j^{r,\ell}$ during generation and replaying it in the training forward pass. Appendix A tabulates every symbol, and

Appendix B separates the discrepancy attributable to version skew, which should be nonzero, from the cross-system component these mechanisms remove.

4.2 TITO: token-in, token-out

Writing $\mathbf{T}_i = \mathbf{p}_i \parallel \mathbf{a}_i$ for the stream turn i contributes, TITO asks that $\mathbf{T}_i$ be a *bit-exact prefix* of $\mathbf{p}_{i+1}$ at every boundary: one displaced identifier replaces $\pi(T_j \mid T_{<j})$ by $\pi(T_j \mid \tilde{T}_{<j})$ at that position and every position after it. Three harness behaviours break the requirement. Encoding is canonical while decoding is many-to-one, so a non-canonical sampled split is lost to its canonical re-encoding; templates prune reasoning before the last User message, which the harness emits once per observation; and a re-serialized tool call returns different whitespace, hence different identifiers.

Token-in preserves prefixes to prevent re-encoding drift. With $\mathcal{M}_i$ the message history, $\mathbf{p}_i = \text{enc}(\text{template}(\mathcal{M}_i))$ and $(\mathbf{a}_i, \mathbf{q}_i) = \text{SAMPLE}_{\pi_{i-1}^r}(\mathbf{p}_i)$ comes from the sampler’s per-token output, so enc acts once per turn rather than once per history replay, with templates pinned to keep rendering append-only.

Token-out stitches streams under loss masking. A trial becomes one stream $\mathbf{T} = (T_1, \dots, T_N)$ with mask $\mathbf{m}$ and log-probabilities $\mathbf{q}$ satisfying

$$m_j = 1 \iff T_j \text{ was sampled by } \pi_{t-1}^r, \quad q_j = m_j \cdot \log \pi_{t-1}^r(T_j \mid T_{<j}, \mathbb{I}_j^r), \quad (5)$$

so observations and glue give context but no gradient. Between turns the assembler tests progressively weaker prefix relations between $\mathbf{T}_i$ and $\mathbf{p}_{i+1}$, and the case it lands in, enumerated in Box 1, determines how the boundary is repaired.

TITO Hierarchy. Boundary cases evaluated at each turn from strongest to weakest.

Each line pairs the condition under which turn i may be appended with what the assembler then appends, the first admissible one deciding the boundary.

$$\text{STRICT} : \mathbf{T}_i \preceq \mathbf{p}_{i+1}, \quad \text{context} = \mathbf{p}_{i+1} \ominus \mathbf{T}_i; \quad (6)$$

$$\text{NORMALIZED} : \min_{(s,u)}(s+u) \text{ s.t. } \text{drop}_s(\mathbf{p}_i) \parallel \text{drop}_u(\mathbf{a}_i) \preceq \mathbf{p}_{i+1}, \quad (s, u) \in [0, 96] \times [0, 16]; \quad (7)$$

$$\text{RETOKENIZED} : \text{dec}(\mathbf{a}_i) = \text{dec}(\tilde{\mathbf{a}}_i), \quad \tilde{\mathbf{a}}_i = \mathbf{p}_{i+1}[b:e] \text{ by offset mapping}; \quad (8)$$

$$\text{SPLIT} : \text{neither holds, so a new chunk opens under the same trial.} \quad (9)$$

Case 6 is exact TITO; Case 7 is a bounded repair on a finite 97×17 grid whose 96 is the shared template suffix, which keeps it auditable. Case 8 would break Equation 3 wholesale, so the assembler falls back to text-space equality and appends

$$\mathbf{T} += \underbrace{\mathbf{a}_i}_{m=1} \parallel \underbrace{\mathbf{p}_{i+1}[e]}_{m=0}, \quad \text{never } \tilde{\mathbf{a}}_i. \quad (10)$$

Empirical verification confirms zero drift. An auditor re-locates $\mathbf{a}_i$ inside $\mathbf{p}_{i+1}$ per transition and checks span and text equality, the right test since Case 8 makes token equality flag harmless re-encodings. Splitting the N positions into aligned $\mathcal{A}$, re-tokenized $\mathcal{D}$ and placeholder $\mathcal{P}$, a production run over 1,402 samples puts the drift rate inside the loss region at **exactly zero**:

$$\frac{|\{j \in \mathcal{D} \cup \mathcal{P} : m_j = 1\}|}{|\{j : m_j = 1\}|} = \mathbf{0.0000}\%, \quad (11)$$

so every position that carries gradient satisfies Equation 3 exactly.

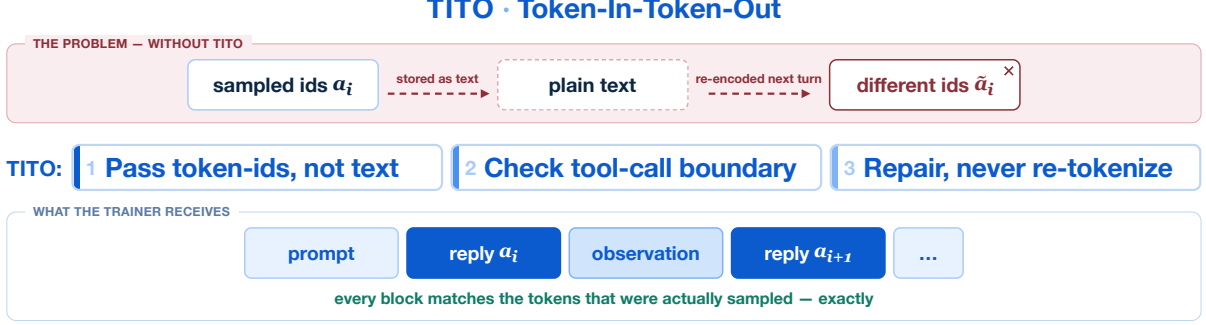


Figure 5 TITO stitching, drawn for the RETOKENIZED case (Equation 8), the hardest of the four. The re-tokenized copy $\tilde{a}_i$ never enters the training stream, whereas the sampled a_i does; observations and glue enter masked, with $m=0$ and $q=0$.

4.3 R^3 : rollout routing replay

R^3 [12] enforces Equation 4 by construction: record the routing mask inference selected, then reuse it in the training forward pass. The primitives are available upstream; we supply the capture path, the multi-turn alignment and the failure policy.

Rollout routing replay. Conventionally the training pass derives both quantities of Equation 2 from its own logits, selecting $\text{TopK}_k(s_j^\ell(\theta_t))$ and normalizing over that selection. R^3 keeps the normalization on the live logits but takes the selection from the recorded mask $\mathbb{I}_j^{r,\ell}$, renormalizing over the recorded experts alone,

$$g_{j,e}^\ell = \frac{\exp([s_j^\ell(\theta_t)]_e)}{\sum_{e' \in \mathbb{I}_j^{r,\ell}} \exp([s_j^\ell(\theta_t)]_{e'})} \text{ for } e \in \mathbb{I}_j^{r,\ell}, \quad \mathbf{y}_j^\ell = \sum_{e \in \mathbb{I}_j^{r,\ell}} g_{j,e}^\ell f_e(\mathbf{h}_j^\ell). \quad (12)$$

This serves two purposes. It *aligns* training with inference, since the experts carrying gradient are exactly those that produced the sample, removing the discontinuity that made an exchanged expert possible. It also *preserves the gradient path*: only the mask is replayed while the softmax still acts on $s_j^\ell(\theta_t)$, leaving the router trainable and the computation graph untouched. The trainer’s router is wrapped, not reimplemented. Replay is only as good as the record, however, and Box 2 states what keeping one costs.

Cost Analysis. Negligible rollout overhead with zero additional arithmetic.

The sampler returns a compact integer tensor $\mathbf{R}_i \in [E]^{\rho_i \times L \times k}$ per turn, one row per predicting position:

$$\mathbf{R}_i[j, \ell, :] = \mathbb{I}_j^{r,\ell}, \quad \rho_i = |\mathbf{T}_i| - 1, \quad b_{\text{tok}} = Lk \cdot 4 \text{ B} = \mathbf{1536} \text{ B/token}, \quad (13)$$

the -1 because the final token predicts nothing. At $(L, k) = (48, 8)$ this is 1.5 KiB per position, or some 48 MiB for a 33k-token trajectory, and it holds the rollout overhead below 3%: the masks are a by-product of a forward pass that already computed them, so capture adds transport but no arithmetic.

Mask caching and multi-turn alignment. Recorded masks inherit the property that makes prefix caching sound: for identical prefix tokens the router yields identical selections, so masks are cached alongside the key-value cache and reused on a prefix hit. This matters because every tool call resumes a shared prefix, and re-prefilling purely to regenerate masks would dominate a long trajectory. On the training side the records ride the same stitching as tokens, keeping $\mathbf{R}[j, \ell, :] = \mathbb{I}_j^{r,\ell}$ at the index j that also indexes (T_j, m_j, q_j) in Equation 5. At repaired boundaries a record may belong to no turn’s capture, and a neighbour is substituted only where no gradient is touched:

$$\mathbf{R}[j, :, :] \leftarrow \mathbf{R}[j-1, :, :] \quad \text{iff } m_{j+1} = 0, \quad \text{otherwise raise,} \quad (14)$$



Figure 6 Train-inference log-probability gap $|\Delta \log p|$ (Equation 15). (Left) Per-step series over the production dense-reward run, with an exponential moving average at $\alpha=0.25$ in bold. (Right) Mean gap on the same 122B stack with and without the two mechanisms.

so replayed routing is approximate only on the set Equation 11 already excludes from the loss. Absent routing is a hard error and the rollout aborts rather than dropping the sample, which would condition the batch on capture having succeeded. Records also receive exactly the token pipeline’s sharding, since any other composition would silently replay the wrong experts, which is why $\rho = N - 1$ is asserted per sample; Appendix B.3 gives the invariant chain.

Algorithm 1 gives the schedule, with two consequences. The reference pass selects freely, so $D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})$ is positive at step 0 by design and any assertion of a vanishing initial divergence must be disabled. Separate forward and backward cursors are needed, since activation recomputation re-runs each layer’s forward pass during the backward pass. The critic never replays, its target requiring no behavioral fidelity to the sampler.

Algorithm 1. Routing-replay schedule for one training step. The router operates in three modes: free selection, replay without gradient, and replay with gradient.

Require: per-turn captures $\{(p_i, a_i, q_i, \mathbf{R}_i)\}_{i=1}^T$ from the sampler

- 1: **for** each turn i **do**
- 2: $(a_i, q_i) \leftarrow \text{SAMPLE}_{\pi_{t-1}^r}(p_i)$ **with** $\mathbf{R}_i[j, \ell, :] \leftarrow \mathbb{I}_j^{\text{r}, \ell}$ ▷ Equation 13
- 3: **end for**
- 4: $(T, m, q) \leftarrow \text{STITCH}(\cdot)$ under cases 6–9; substitute a record only where $m_{j+1} = 0$ ▷ Equation 14
- 5: **abort** if any capture is missing; **assert** $\rho = |T| - 1$; shard $\mathbf{R}$ as T
- 6: **free:** evaluate $\log \pi_{\theta_{\text{ref}}}(T)$ ▷ $D_{\text{KL}} > 0$ at step 0
- 7: **replay, with gradient:** $\theta \leftarrow \theta - \eta_\theta \nabla_\theta \mathcal{L}^{\text{PPO}}$ with $r_j(\theta)$ from Equation 1
- 8: **free:** $\phi \leftarrow \phi - \eta_\phi \nabla_\phi \mathcal{L}^V$; release the record

4.4 Measuring Training Stability of T1

Measure the Training-Inference Mismatch. Following Yang et al. [30], the realized mismatch at a loss-bearing position is the log of the sampled importance ratio:

$$d_j = \log \pi_t - \log \pi_{t-1}. \quad (15)$$

At a staleness of one update the first term is the single optimizer step the clip is designed to correct, and it should be nonzero; the second is nonzero even at identical weights, because the two stacks differ in kernels, reduction orders and routing, and it is precisely what TITO and R³ remove. As shown in Figure 6, we report the mask-weighted mean of $|d_j|$ over the loss region, computed outside the autograd path so the recipe is bit-identical whether or not it is collected. The reduction matters: normalizing by micro-batch count rather than by $\sum_j m_j$ inflates the statistic by three orders of magnitude at unit micro-batch.

T1 training stability. Each mechanism pairs with a quantity that certifies it. Token fidelity is confirmed by Equation 11, which puts drift inside $\{j : m_j = 1\}$ at 0.0000%. Routing fidelity is enforced by substitution rather than measured, since Equation 12 replaces TopK_k by a lookup, leaving only the placeholder set of Equation 14, bounded at 0.003% and disjoint from the loss region. Jointly they move the gap from 0.021 to 0.013, which decides whether $r_j(\theta)$ reflects policy movement or bookkeeping error. The residual is expected, as R^3 aligns expert selection but not kernel numerics, and a slow upward drift is no regression either, since Δ_j^π grows when the policy legitimately improves.

Algorithmic measures that keep the run stable and efficient. We keep the objective deliberately spare and bound the cost of a single step, withholding every optional term that fought an alignment mechanism or added a gradient the reward does not justify:

- **Surrogate clipping & KL penalties:** The surrogate clips symmetrically at $\varepsilon = 0.2$, while both KL terms are disabled because the frozen reference routes with its own selection and would unfairly charge the policy for a bookkeeping difference.
- **MoE load balancing:** The load-balancing coefficient is set to **zero**, as balancing pressure asks the router to redistribute exactly the choices replay asks it to reproduce.
- **Optimizer configuration:** Optimization is performed using Adam with $\beta = (0.9, 0.98)$, weight decay of 0.1, and a constant learning rate.
- **Oversampling:** Long-horizon trials have a heavy length tail, so a step that waits for every trajectory is paced by its slowest few. We therefore train at a batch of **560** and oversample during rollout, admitting the first **512** trajectories to complete and utilize data parallel at **8** to accelerate training, truncating the remaining tail, which bounds step time at the price of a mild bias against the longest trials.
- **Compilation storms:** A compilation storm is indistinguishable from a collective hang, one rank having once compiled for over 30 minutes while its peers waited inside the all-to-all.

The scheduling and liveness machinery that realizes the last two items is described in Section 7.

4.5 Critic-side stability

PPO here uses a separate, full-size critic: a second copy of the same architecture whose language-model head is replaced by a scalar value head on the last pipeline stage. It shares the actor’s device allocation without additional hardware, and training-side offload is forced so that the two 122B networks time-multiplex those devices. Each must therefore fit alone in 95 GiB, which is the constraint driving Section 7.1.

Ordering. The critic trains first each step and hands its pre-update values $V_{\text{old}} = V_{\phi_{t-1}}$ to the actor, anchoring both the advantage estimator and the value objective to one fixed function:

$$\hat{A}_j = \sum_{n \geq 0} (\gamma \lambda)^n \delta_{j+n}, \quad \text{where} \quad \delta_j = \hat{r}_j + \gamma V_{\text{old}}(s_{j+1}) - V_{\text{old}}(s_j), \quad (16)$$

$$\mathcal{L}^V(\phi) = \mathbb{E}_j \left[\max \left((V_\phi(s_j) - \hat{R}_j)^2, (V_{\text{clip}}(s_j) - \hat{R}_j)^2 \right) \right], \quad (17)$$

$$\mathcal{L}^\pi(\theta) = -\mathbb{E}_j \left[\min \left(r_j(\theta) \hat{A}_j, \text{clip}(r_j(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_j \right) \right], \quad (18)$$

where $V_{\text{clip}}(s_j) = V_{\text{old}}(s_j) + \text{clip}(V_\phi(s_j) - V_{\text{old}}(s_j), -\epsilon_v, \epsilon_v)$ and $r_j(\theta) = \frac{\pi_\theta(a_j|s_j)}{\pi_{\theta_{\text{old}}}(a_j|s_j)}$ denotes the probability ratio. Here, $\gamma = \lambda = 1$, and the clipping thresholds are set to $\epsilon = 0.2$ for the actor policy and $\epsilon_v = 0.2$ for the critic value function.

Critic Warm-Up. Critic Warm-Up trains the value network as well as the actor, with only critic model being saved, for one epoch over TMax-15k before any policy step is taken. The dense-reward campaign loads those weights, weights only so that no optimizer moment crosses runs, and needs just $N=2$ re-calibration rollouts, whereas the binary-reward campaign cold-started its critic instead.

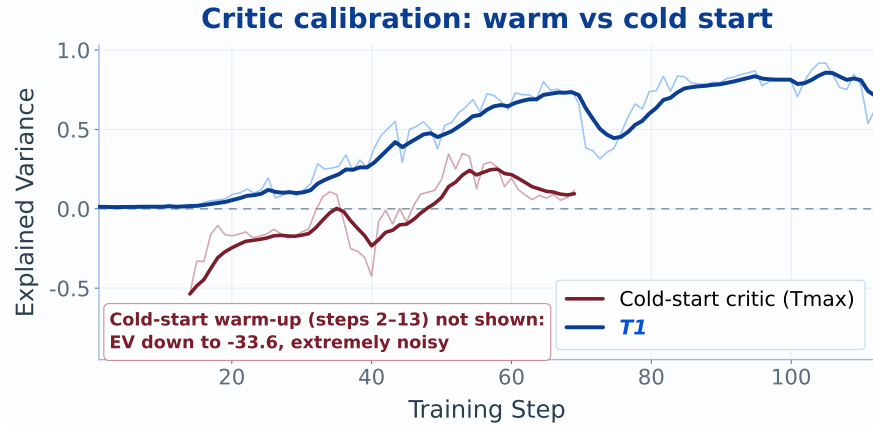


Figure 7 Critic explained variance, defined in Equation 19, with and without Critic Warm-Up. **Blue (T1)**: the production dense-reward run on T1-15k, whose critic comes from Critic Warm-Up over TMax-15k. **Red**: the cold-started critic of the TMax-15k campaign. Faint lines are per-step values and bold lines an exponential moving average at $\alpha=0.25$, while the dashed rule marks $EV=0$. The cold start starts at $EV=-33.6$ and is negative for 30 of 58 logged steps, whereas the Critic Warm-Up run never goes negative and plateaus between 0.71 and 0.86.

What this buys is read off explained variance, the fraction of return variance the value function accounts for, computed over the tokens of a rollout batch with GAE returns $\hat{R}_j$:

$$EV = 1 - \frac{\text{Var}_j[\hat{R}_j - V_\phi(s_j)]}{\text{Var}_j[\hat{R}_j]}, \quad (19)$$

where both variances are reduced globally across context- and data-parallel ranks, since a per-rank value averaged across ranks is biased. The quantity is unbounded below, and $EV < 0$ means that subtracting V_ϕ adds variance to $\hat{A}_j$ instead of removing it. A cold start opens at -33.6 and spends roughly the first half of the campaign paying down that deficit, whereas Critic Warm-Up settles between 0.71 and 0.86 from the first update. The same measurement fixes the learning rates: the critic runs at 1.5×10^{-5} against 1.0×10^{-6} for the actor, absorbing the larger step because its target is a supervised regression rather than a policy improvement, and on 27B pathfinding runs moving this ratio from $10\times$ to $20\times$ lifted EV from -39 to $+0.11$ while $30\times$ shortened Critic Warm-Up further. Reward stayed flat across these settings, which redirected attention to the data as Section 5 describes.

Value-target conditioning. Two choices keep the regression target well-posed. The dense reward uses a global fixed scale, as Section 5.2 sets out, and the trajectory reward lands on the final token with $\gamma = \lambda = 1$, which makes per-token returns piecewise constant so that the value problem reduces to predicting a trajectory’s final score from its prefix.

5 Dense Verification Reward Design

5.1 From sparse outcomes to dense verification signals

When we began RL on T1-15k, many tasks were too difficult for the model to solve completely. Under a binary task-solved reward, these unsuccessful trajectories all received zero, even when the agent had satisfied some of the task’s requirements. Complete successes were too rare to provide a useful learning signal, while partial progress remained invisible to the reward. Learning on these tasks therefore required feedback that could distinguish degrees of completion before the model could reliably produce a full solution.

Such feedback depends on the verifiers supplied with the training data. In our earlier RST work, we anticipated this need during task synthesis and equipped the synthesized tasks with sufficiently many verification checks covering individual task requirements. These checks make partial completion observable through per-assertion

outcomes. By comparison, TMax-15k lacks a sufficiently rich set of verification checks to support this form of dense reward. The RST synthesis process thus provides the foundation for dense feedback in our synthesized training pools, including T1-15k.

We use these verification outcomes to reward the number of assertions an agent satisfies, giving credit for partial solutions even when the overall task remains unsolved. This turns otherwise zero-reward trajectories into graded supervision and allows the model to learn from progress on tasks it cannot yet complete. The following subsection defines how these per-assertion outcomes are converted into a scalar reward.

5.2 Per-assertion verification and the test-count reward

Each task’s verifier is invoked so that it emits a structured per-assertion report inside the sandbox. After rollout the reward stage reads that report back from the trial and computes, with P the number of passing assertions,

$$r = \frac{P}{S}, \quad S = 20, \quad (20)$$

that is the absolute passing count on a fixed global scale, and explicitly not the pass ratio. Three decisions are worth stating.

- **Absolute count rather than ratio.** Our training pool mixes tasks of different difficulty, and we shuffle them together without an easy-to-hard curriculum. A single batch therefore contains both easy and hard tasks, making the reward scale across tasks consequential. For example, passing 10 of 20 assertions on a hard task and passing 2 of 4 on an easy task both yield a pass ratio of 0.5. Yet satisfying those ten assertions can require substantially more work, potentially through a long sequence of tool interactions. The equal ratios hide this difference in verified progress. Our synthesis process was designed with this comparison in mind: harder tasks were equipped with more verification checks, while each check was intended to represent a roughly comparable increment of work across tasks. This is an approximate design principle, rather than a guarantee that all assertions require identical effort. Under this principle, the absolute passing count better reflects the amount of verified progress in a mixed batch. With the shared scale $S = 20$, the two trajectories above receive $10/20 = 0.5$ and $2/20 = 0.1$, respectively. Each additional passing assertion contributes the same $1/S$ reward, preserving the intended distinction between completing more requirements on a hard task and fewer on an easy one.
- **Fixed global scale.** We chose $S = 20$ after measuring the assertion-count distribution across the 15,000 tasks in T1-15k. This value sits near the ninetieth percentile of that distribution, whose median is 4 and maximum roughly 35. We keep it fixed throughout training: a per-batch maximum would make the reward scale drift from step to step and hand the critic an inconsistent regression target, whereas cross-step consistency is precisely what makes the value function learnable.
- **Fallback.** If the per-assertion report is missing or unparsable the trial falls back to the binary terminal outcome, so a genuinely solved task never scores zero, and parse failures are tagged by cause for observability.

An earlier ratio-based variant, $r = \max(b, 0.4 P/T)$ clamped to $[0, \max(1, b)]$ with T the total assertion count and b the terminal outcome, is retained for controlled comparison. Algorithm 2 states the computation as used in production.

Figure 8 shows the reward this definition actually produces in training, and it is worth reading against the design choices above. The absolute scale behaves as intended: a mean near 0.35 at $S=20$ corresponds to roughly 7 passing assertions per trajectory, comfortably inside the resolution of the signal rather than pinned at either end, which is precisely what the ratio formulation would have destroyed by capping every task at 1.0. The trajectory has two phases, a steep climb from 0.250 to roughly 0.345 over the first 50 steps and then a plateau in the band 0.34 to 0.36 for the remaining 60 steps, peaking at 0.365 near step 58. The plateau is not stagnation, since the held-out benchmark keeps improving through it as Section 6.3 shows, so the run redistributes which assertions it satisfies rather than simply satisfying more of them. This is consistent with per-epoch reshuffling keeping the per-batch reward statistics stationary rather than letting them drift with

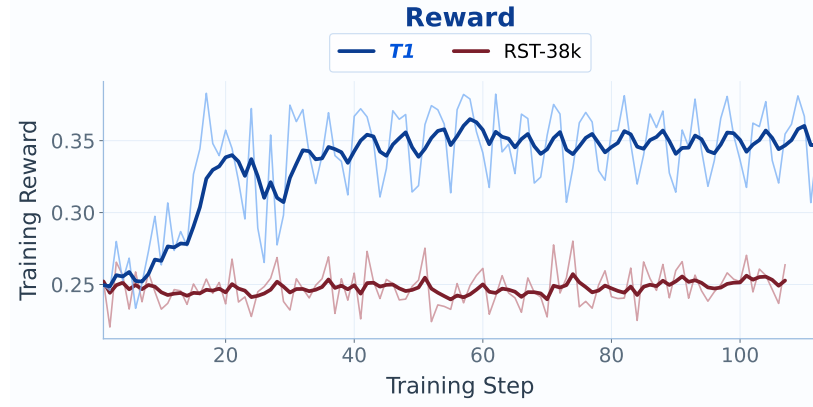


Figure 8 Mean rollout reward under the test-count reward over the production dense-reward run, reported before any normalization, since under PPO the reward stage returns raw values as Algorithm 2 states. Faint line: per-step mean; bold line: exponential moving average at $\alpha=0.25$. Reward rises from 0.250 to roughly 0.345 in the first 50 steps and then holds a band of 0.34 to 0.36 for the remaining 60.

task quality. The visible high-frequency oscillation is the batch-composition signature of that shuffling at batch 512 with one sample per task rather than an instability: its amplitude of about 0.02 stays constant over the run, and the band never collapses toward zero.

Algorithm 2. Dense reward assignment for one rollout step. P is the number of passing assertions reported by the verifier, $b \in \{0, 1\}$ the terminal outcome, and S the fixed normalizer.

Require: completed trials of the current step, grouped by task and trajectory

```

1: for each trial do
2:    $b \leftarrow$  terminal verifier outcome
3:   if a per-assertion record is available then
4:      $r \leftarrow P/S$   $\triangleright S = 20$ ;  $r$  may exceed 1
5:   else
6:      $r \leftarrow b$   $\triangleright$  a solved task is never assigned zero
7:   end if
8:   broadcast  $r$  to every chunk of the trial, so credit is trajectory-level
9: end for
10: return  $r$  unnormalized, since with one sample per task no group baseline exists
11: place  $r$  on the final response token; GAE with  $\gamma = \lambda = 1$  against the critic distributes credit over the horizon

```

5.3 Credit assignment

Density in Equation 20 concerns the reward’s value resolution rather than its temporal placement: the score gains $1/S$ per additional satisfied assertion, yet is still delivered as one scalar on the last response token of each trajectory chunk. This formulation has two direct implications for learning:

- **Temporal credit comes from the value function.** GAE at $\gamma = \lambda = 1$ propagates the terminal scalar backward over the interaction horizon, and no potential-based shaping or per-turn term is added.
- **The reward stays unnormalized.** Each task contributes one trajectory per step, so no group statistic exists from which a group-relative baseline could be formed, and advantage normalization would rescale away the cross-task differences an absolute passing count is meant to preserve.

The critic is thus the only baseline, and its calibration in Section 4.5 therefore lies on the critical path.

5.4 Preventing reward hacking

We address reward hacking at two levels: the tasks admitted to the training pool and the incentives created by the reward function.

Data-side filtering: rejecting exploitable tasks. Our primary defence is to reject tasks that can be gamed before they enter the training pool. The T1-15k pool is selected through a semantic audit with DeepSeek-V4-Pro. Tasks receive a `hard_reject` for any of four problems: hidden requirements, test leakage, solution shortcuts, or verifiers too weak to validate the task’s goal (Section 3.1). This filtering aims to remove opportunities to earn reward without accomplishing the intended task, and is our only defence directed at the exploitability of the tasks themselves.

Reward-side design: monitoring turn growth and fixing the scale. The test-count reward can create an incentive to prolong trajectories simply to pass more tests, leading to uncontrolled growth in turn counts. We observed this behaviour in 27B experiments and investigated various length-shaping variants [8]. For the 122B run, we closely monitored turn counts and sequence lengths and did not observe the same runaway growth (Figure 13). The default 122B run uses the plain test-count reward, with length shaping disabled.

We also keep $S = 20$ fixed globally, using the value selected from the T1-15k statistics in Section 5.2. With normalization by a per-batch maximum, a single extreme sample could change the denominator and hence the reward scale for the entire batch. A fixed denominator makes each trajectory’s reward independent of the other samples’ passing counts, removing this route for manipulating the batch’s reward scale.

Data interaction: shuffling is part of the reward design. The T1-15k pool is materialized in quality-rank order. Without shuffling a sequential cursor would sweep from best to worst, at the batch size of 512 tasks would be drawn from one narrow quality band and the reward distribution the critic sees would drift monotonically over the epoch.

6 Results

6.1 Benchmark and Harness

We evaluate on three held-out suites, none of which contributes tasks to any training pool, so that breadth, horizon length and raw difficulty are measured separately.

- **Terminal-Bench 2.1** [14] is our primary held-out benchmark for terminal agent capability: 89 tasks graded by execution and reported as the fraction resolved. It supersedes Terminal-Bench 2.0, whose instabilities hindered reproducible evaluation and underestimated benchmark performance.
- **Long-Horizon Terminal Bench (LHTB)** [9] is a suite of 46 hard, reproducible tasks across nine categories, designed to resist memorization, shortcutting and reward hacking. Every task pays continuous partial credit instead of binary pass or fail, so we report average reward rather than a resolved rate.
- **Terminal-Bench Hard (TBH)** [8] is a 100-task evaluation set and reported as the fraction resolved. It probes an independently constructed and harder task distribution, and is distinct from the Hard difficulty group inside Terminal-Bench 2.1.

All three suites run under one configuration. The agent is Terminus-2, hosted by Harbor: a structured tool-call loop in which each assistant turn issues shell commands and each tool turn returns terminal output, permitting at most 60 turns under an agent wall of 3600s and a verifier wall of 900s. Explicit thinking is disabled in the chat template, proactive context compaction is enabled so the agent summarizes its own history once the context reaches a configured threshold, and every trial receives a freshly created cloud sandbox of 10 GiB disk that is destroyed on completion. Decoding uses a context window of 96,000 tokens at temperature 0.1, top- p 0.95 and top- k 20, with 3 attempts per task. The evaluation-total timeout is set to 18,000s. Two starting checkpoints appear across our campaigns, the base model at 43.8% and an SFT checkpoint denoted RST at 49.4%, the latter produced by rejection-sampling-style fine-tuning outside our scope.

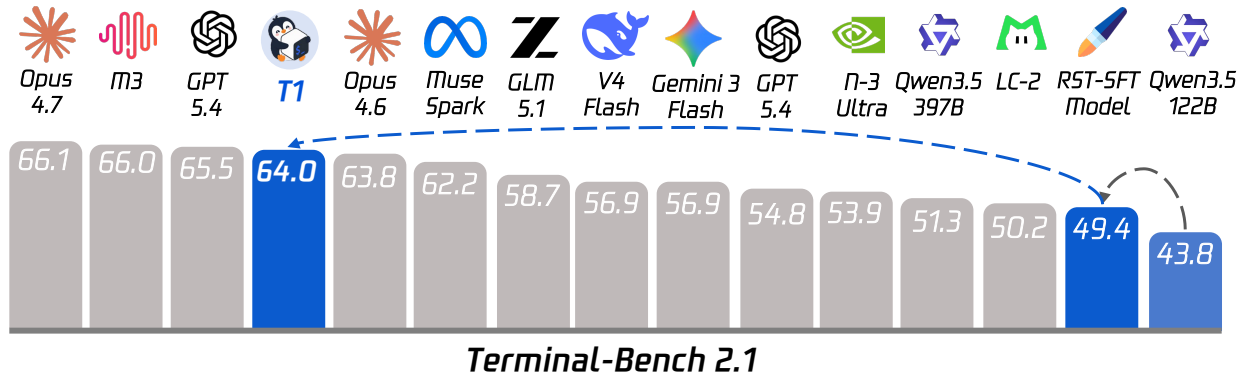


Figure 9 Terminal-Bench 2.1 standing of **T1** against contemporary frontier and open-weight models. Blue bars trace our own pipeline: the Qwen3.5-122B-A10B base model (43.8), the RST-38k SFT checkpoint we initialize from (49.4), and **T1** after RL (64.0); the dashed arrows mark the two stages, a gain of 5.6 percentage points from SFT and a further 14.6 percentage points from reinforcement learning. Grey bars are the comparison models. With 10B active parameters **T1** ranks fourth overall and ahead of Claude Opus 4.6, and is the only model in the leading group that reaches that band from a sub-50 starting point.

Training and evaluation share one software stack. We build our distributed training and rollout framework on top of slime v0.3.0, with Megatron-Core v0.16.0rc0 and Transformer Engine v2.10.0 on the training side, utilizing SGLang v0.5.12.post1 together with sglang-kernel v0.4.2.post2 and DeepEP v1.2.1 as the rollout inference engine with vendor-specific synchronization and KV-cache optimization patches. Trials are executed through Harbor v0.7.0 against Daytona sandboxes version v0.168.0. The underlying runtime is PyTorch 2.11.0 with CUDA 12.9 and NCCL 2.28.9, and orchestration uses Ray 2.55.1.

6.2 Main results

As shown in Table 1, we evaluate the model on Terminal-Bench 2.1 and Long-Horizon Terminal-Bench (LHTB). Shown in Figure 11, we evaluate the model on Terminal-Bench Hard for more challenge terminal tasks.

Binary reward on TMax-15k, from the base model. Our training is based on both critic model and actor model as Qwen3.5-122B-A10B without warm-up, using Tmax-15k as our training dataset. This campaign improves the base model early, from 43.8% to 47.2% at iteration 30, but never reaches the SFT checkpoint’s 49.4%. Its lasting contribution is the trained critic of Section 4.5.

Dense reward on the unfiltered RST-38k, from the SFT checkpoint. Our training is based on the warm-up critic model and the RST-SFT model as training initial actor model on RST-38k as our training dataset. The model reaches immediately above RST-SFT Model, reaching 59.9% at iteration 70.

Dense reward on T1-15k, from the SFT checkpoint after Critic Warm-Up. This is the production run of the present report: PPO at batch 512 with 84k context length, routing replay enabled, and a critic obtained by Critic Warm-Up over TMax-15k for one epoch. Our training is based on the warm-up critic model and the RST-SFT model as training initial actor model on T1-15k as our training dataset. At step 110 repeated evaluations reaches the best performance of **64.0%**.

Under the same Terminus-2 harness and Daytona sandbox backend, reported in Table 1, the RL checkpoint sits above GLM-5.1, Hy3-Preview, DeepSeek-V4-Flash, Kimi-K2.5, Minimax M2.7, GPT-5.4 and Claude Sonnet 4.6, and within two points of Claude Opus 4.6, while using 10B active parameters. We attach the standard caveat that harness and constraint choices materially move these numbers, as discussed below, and that our model was RL-trained for exactly this harness whereas the frontier models were not.

Figure 9 puts that standing next to the trajectory that produced it, which is the part a leaderboard row hides.

Table 1 Performance Comparison. The upper block lists rows evaluated under the same harness as ours, while the lower block gives selected public leaderboard rows obtained under other harnesses and shown for context only, since harness choice materially changes scores: Claude Opus 4.6 scores 70.1 under Claude Code against 63.8 under Terminus-2. Sizes are as recorded in the evaluation workbook, and a solidus denotes a size not recorded. The rightmost column reports average reward on Long-Horizon Terminal-Bench (LHTB).

Model	Size (total-active)	Terminal-Bench 2.1	LHTB
<i>Same harness (Harbor/Terminus-2):</i>			
Claude Opus 4.7	/	66.1	–
Claude Opus 4.6	/	63.8	–
Muse Spark	/	62.2	–
Hy3-Preview	295B-A21B	58.0	–
DeepSeek V4 Flash (high)	295B-A21B	56.9	–
Kimi-K2.5	1040B-A32B	56.4	–
Minimax M2.7	229B-A10B	55.4	–
GPT-5.4	/	54.8	27.2
Gemini 3 Flash	/	54.2	–
Claude Sonnet 4.6	/	51.5	37.3
Qwen3.5-122B-A10B (base)	122B-A10B	43.8	18.9
RST-SFT Model	122B-A10B	49.4	23.6
Qwen3.5-122B-A10B + RL (Tmax-15k)	122B-A10B	47.2	20.3
Qwen3.5-122B-A10B + RL (RST-38k)	122B-A10B	59.9	25.4
T1	122B-A10B	64.0	27.9
<i>Other harnesses (public leaderboards, context only):</i>			
GPT-5.3-Codex (Codex CLI)	/	79.1	21.5
GPT-5.4 (Codex CLI)	/	77.3	27.2
Claude Opus 4.6 (Claude Code)	/	70.1	–
Gemini 3.1 Pro (Gemini Code)	/	67.1	27.9
GLM-5.1 (Claude Code)	750B-A40B	58.7	26.7

Our three blue bars are the same model at three stages: Qwen3.5-122B-A10B base at 43.8, the RST-38k SFT checkpoint at 49.4, and **T1** at 64.0. Read left to right, the RL stage moves the model past nine of the comparison systems in a single step. Supervised fine-tuning alone leaves it second from last, below every comparison entry in the chart, while the same weights after RL post-training land fourth overall and above Claude Opus 4.6. The contrast in step sizes is the substantive claim: SFT contributes 5.6 percentage points and RL a further 14.6 percentage points, so roughly three-quarters of the total distance from base to final is earned by reinforcement learning on terminal tasks rather than by imitation of demonstrations. It is also worth noting what the bars do not encode. The models above and immediately below us are dense or far larger sparse systems, whereas **T1** reaches this band with 10B active parameters, which restates the efficiency argument of Section 1 in benchmark terms.

Transfer to longer horizons. Long-Horizon Terminal Bench tests whether the gains on Terminal-Bench 2.1 extend to tasks that require more sustained interaction with the environment. Figure 10 shows a consistent improvement across our three checkpoints: the base model scores 18.9, SFT raises this to 23.6, and RL reaches **27.9**. The RL stage therefore adds **4.3 average-reward points** over SFT, an 18.2% relative improvement; the full pipeline gains 9.0 points over the base model. The improvement continues beyond the earlier T1-15k checkpoint at iteration 70 (25.5) and the RST-38k RL model (25.4) in Table 1, indicating that the stronger Terminal-Bench 2.1 result is accompanied by progress on the longer-horizon evaluation.

Against the comparison models in Figure 10, **T1** matches Gemini-3.1-Pro at 27.9 and exceeds GPT-5.4 (27.2), GLM-5.1 (26.7), and Kimi-K2.6 (25.5), while GLM-5.2 (31.6) and DeepSeek-V4-Pro (30.7) remain ahead. These results place a model with 10B active parameters in a competitive band on this evaluation. The comparison is not uniform across benchmarks: Claude Sonnet 4.6, which scores below **T1** on Terminal-Bench 2.1, records 37.3 on LHTB in Table 1. Longer-horizon performance therefore warrants a separate evaluation rather than being inferred from the Terminal-Bench 2.1 ordering alone. We report LHTB in its

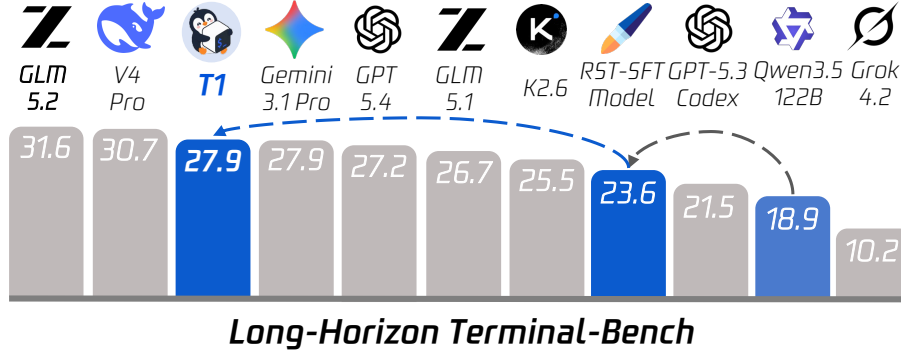


Figure 10 Performance on Long-Horizon Terminal Bench. The base model, RST-SFT checkpoint, and **T1** score 18.9, 23.6, and 27.9, respectively. RL adds 4.3 average-reward points over SFT, and **T1** matches Gemini-3.1-Pro among the models shown.

average-reward units, separately from the resolved percentages of the other benchmarks.

Generalization to harder terminal tasks. Figure 11 reports the complementary evaluation on Terminal-Bench Hard (TBH). **T1** resolves **38.0%** of tasks, compared with 28.3% for the SFT checkpoint and 20.0% for the base model. RL contributes **9.7 percentage points** beyond SFT, a 34.3% relative improvement, while the full pipeline gains 18.0 percentage points over base. The final score also exceeds DeepSeek-V4-Pro at 36.0% by 2.0 percentage points. Together with the LHTB results, this supports the view that terminal-agent post-training improves performance as both task difficulty and interaction horizon increase. The comparison concerns the complete training recipe; it does not isolate the contribution of dense reward from the data pool or the stabilization mechanisms.

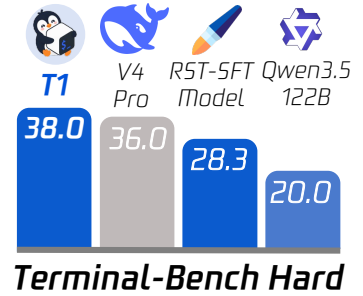


Figure 11 Terminal-Bench Hard resolved rate: **T1** outperforms the RST-SFT checkpoint by 9.7 points and the base model by 18.0 points.

6.3 Training dynamics

Figure 12 traces the held-out Terminal-Bench 2.1 resolved rate of **T1** as a function of training step, evaluated every ten rollout steps against the two fixed anchors of the run, namely the base model at **43.8%** and the SFT checkpoint at **49.4%** from which the run is initialized. The first evaluated checkpoint already reaches **56.2% at step 10**, exceeding that initialization by **6.8 percentage points**, which is consistent with the reward curve of Figure 8 rising fastest over the same interval. Steps 20–60 then hold a band of **55.1–57.3%**, a spread of roughly three tasks out of 89 that is comparable to the variation observed across repeated evaluations of a single checkpoint, and no evaluated checkpoint returns to the initialization. Two further increases arrive late in the run, **61.8% at step 70** and **64.0% at step 110**, the latter standing **14.6 percentage points** above SFT and **20.2 percentage points** above the base model. Both coincide with the region where explained variance in Figure 7 is highest, which indicates that the largest usable actor improvements occur only after the critic is well calibrated.

Figure 13 reports the behaviour that produces those scores. Average tool-call turns per trajectory approximately double, from **10.4** to a plateau near **20.7**, and total sequence length follows from **11.5k** to roughly **18.3k** tokens. Two properties distinguish this trend from length hacking. The growth is bounded, since both curves flatten after step 60 and the final third of training contributes almost nothing, whereas the additive length penalty produced unbounded growth beyond **50 turns**. The plateau is concurrent with the reward and benchmark gains, which indicates that additional turns yield additional passing assertions rather than padding. Turn growth nonetheless contributes to the timeout failures examined in Section 6.4.

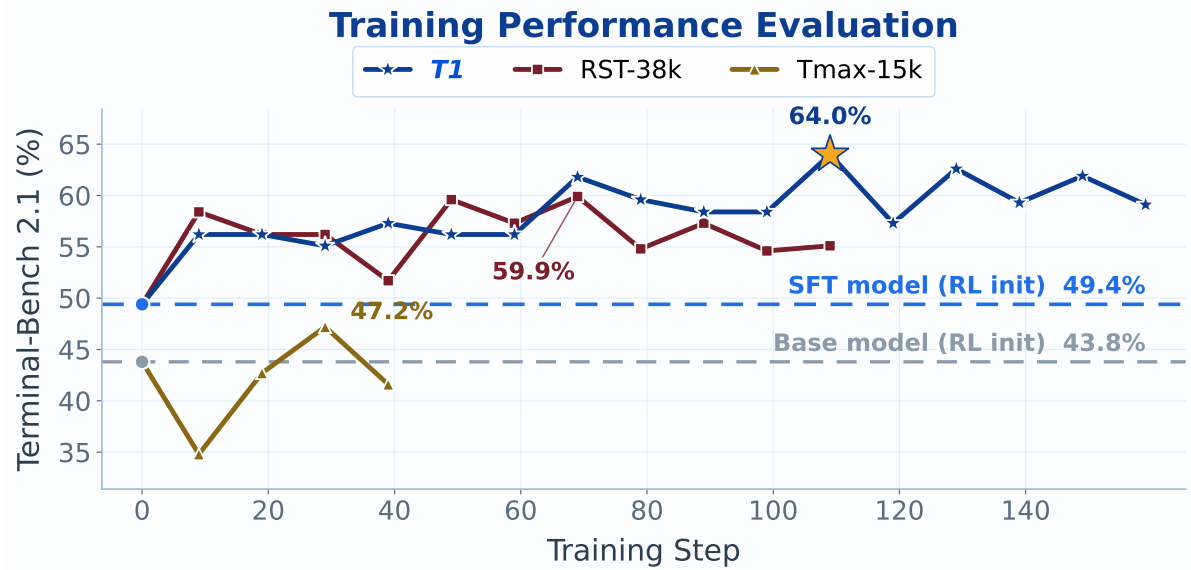


Figure 12 Held-out Terminal-Bench 2.1 resolved rate over the production dense-reward run. Filled markers are the evaluated checkpoints (every 10 rollout steps), each annotated with its score; the starred point is the peak. The dashed rules are the two fixed anchors, namely the base model at 43.8% and the SFT checkpoint the run is initialized from at 49.4%. The very first evaluated checkpoint already clears SFT by 6.8 percentage points, no evaluated checkpoint ever falls back to the initialization, and the peak at step 110 reaches 64.0%, which stands 14.6 percentage points above SFT and 20.2 percentage points above base.

6.4 Case study: detailed evaluation across benchmarks

Where the aggregate gain comes from. Figure 14 decomposes the improvement from the base model to **T1**. SFT raises the resolved rate from 43.8% to 49.4%, a gain of 5.6 percentage points, and RL adds a further **14.6 percentage points** to reach 64.0%. Measured against each stage’s initialization, these are 12.8% and 29.6% relative improvements, respectively. RL thus accounts for 72.3% of the total gain of 20.2 percentage points from base to final, making it the larger contributor in this training pipeline.

The right panel highlights two domains where **T1** is particularly strong. On debugging, it reaches **100.0%** against 80.0% for GPT-5.6 Sol; on system administration, it reaches **88.9%** against 55.6%, gains of 20.0 and 33.3 percentage points, respectively. Both domains require the agent to inspect environment state, act on a working hypothesis, and revise its approach from execution feedback. Evaluate through Wang et al. [24], their results are consistent with the capabilities exercised by our terminal-agent RL loop. The subsets are small, however: debugging contains five tasks and system administration nine, so the domain scores describe specific strengths within this evaluation rather than establishing an overall ordering between the two models.

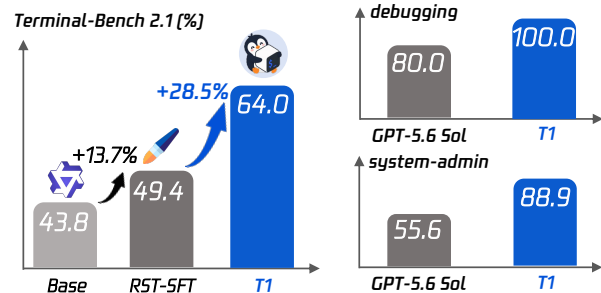


Figure 14 (Left) Terminal-Bench 2.1 progression across the three checkpoints of our pipeline, from base through SFT to **T1**. (Right) per-domain comparison against GPT-5.6 Sol on the two subsets that depend on multi-step reasoning and tool use.

Domain gains are uneven. The full breakdown in Figure 15 shows that the advantage extends beyond the two highlighted domains. **T1** also resolves all tasks in data processing and machine learning, although these categories contain only four and three tasks, respectively. The gains are not universal: GPT-5.6 Sol remains ahead on data science, scientific computing, and mathematics, while file operations remain difficult for our model. This pattern points to domain-specific headroom that the aggregate resolved rate obscures. The

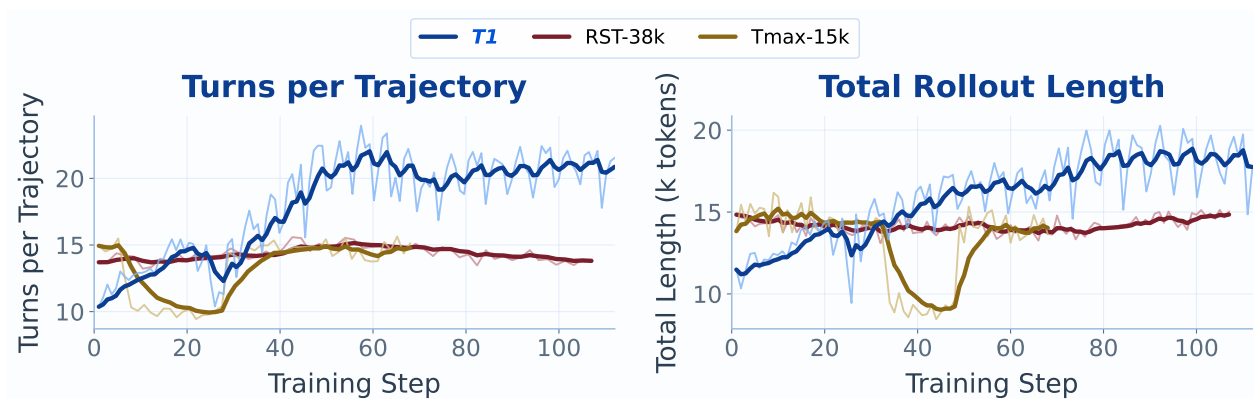


Figure 13 Behavioral dynamics of the dense-reward run. (*Left*) average tool-call turns per trajectory. (*Right*) total sequence length per trajectory. Faint lines are per-step values, bold lines an EMA ($\alpha=0.25$). Both roughly double over training (turns $10.4 \rightarrow 20.9$, length $11.5k \rightarrow 17.8k$ tokens) and then flatten, rather than growing without bound.

concentration of T1-15k in command-line engineering work (Section 3.1) provides one plausible explanation for these differences, but the category comparison alone does not separate data coverage from reasoning and tool-use limitations.

Most of the difficulty-level gain is on Medium tasks. All three checkpoints reach 100% on the Easy group, leaving no measured headroom there. On Medium tasks, **T1** reaches **78%**, compared with 58% for SFT and 56% for base: gains of approximately 20 and 22 percentage points. On Hard tasks, the corresponding scores are **33%**, 30%, and 20%, so the RL-stage gain is smaller at approximately 3 percentage points. These rounded group scores show that RL’s improvement over SFT is largest on Medium tasks, while the hardest group remains a substantial source of failures. The 33% here and the 38.0% on TBH refer to different evaluations and should not be compared as successive checkpoints.

Higher success comes with more interaction. In the evaluation summarized by Figure 15, **T1** uses 94.4 turns on average, compared with 31.5 for SFT and 41.1 for base, approximately $3.0\times$ and $2.3\times$ as many turns. This is consistent with a policy that sustains longer attempts, but the aggregate statistics do not establish whether the additional turns are productive on individual tasks. In particular, a large increase in turns accompanies only a modest improvement on the Hard group. These are evaluation trajectory lengths, distinct from the training averages in Section 6.3; they measure a cost of the resulting policy as well as its capacity for extended interaction.

Failure cases expose inefficient search. The project case study against GPT-5.6 Sol makes this cost concrete. On six representative unsolved tasks, **T1** spends between 164 and 473 turns, compared with 6–40 for GPT-5.6 Sol, and times out; three further tasks fail on sandbox errors. Only four failures reach the 500-turn evaluation ceiling, while eleven record partial sub-test passes. The comparison also uses different resource settings: our maximum input is 56k tokens against 120k, maximum output is 8,192 against 32,768, and explicit thinking is disabled against medium reasoning effort. These differences limit attribution of the failures to model capability alone. Together, the cases suggest that longer interaction is useful only when paired with effective diagnosis, recovery, and stopping decisions; increasing the turn budget alone would not address the observed failure modes. Appendix C examines the complementary direction, contrasting two tasks that **T1** resolves and its SFT initialization does not, to show which behavioural changes the RL stage is responsible for.

Terminal-Bench 2.1 Performance Analysis

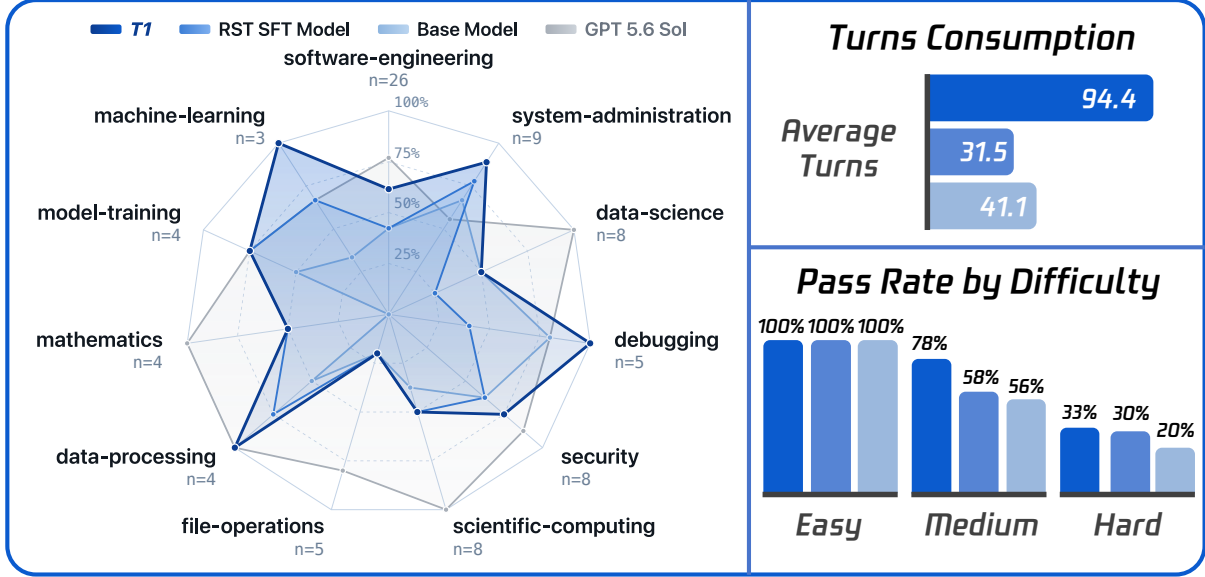


Figure 15 Terminal-Bench 2.1 performance by domain, interaction length, and difficulty. (Left) resolved rates across 11 domains, with task counts on each axis; GPT-5.6 Sol is included as an additional comparison. (Top) average turns for T1, RST-SFT, and base: 94.4, 31.5, and 41.1. (Bottom) resolved rates for the same three checkpoints: all score 100% on Easy tasks, while Medium scores are 78/58/56% and Hard scores are 33/30/20%. Difficulty percentages are rounded as displayed in the figure.

7 Infrastructure

To maximize throughput, we decouple training and inference across disjoint accelerator pools in our pipeline using asynchronous reinforcement learning based on slime as our basic training framework [23]. While this overlap substantially improves training efficiency, it introduces severe infrastructure bottlenecks. Three challenges dominate: co-locating two 122B actor-critic parameter sets under strict device limits, managing hour-scale steps, and scaling out concurrent external sandboxes. We resolve each via principled resource and liveness models rather than heuristic tuning. Throughout, G is the device count, M the per-device memory budget, and a *sharding plan* Π assigns degrees $(\tau, \rho, \kappa, \delta)$ subject to $\tau\rho\kappa\delta = G$.

7.1 A capacity model for co-resident actor-critic pairs

Because the two networks time-multiplex one device set through forced offload (Section 4.5), each must independently satisfy M , so memory rather than arithmetic throughput is the binding constraint. The first joint update on our initial plan aborted inside the critic’s recurrent forward, requesting a further allocation of well under one percent of M on a device already resident above 95% of its budget, after critic-only steps had run cleanly. Rather than search the plan space, we calibrated a closed-form per-device footprint, with $P_e = 116.0\text{B}$ expert and $P_d = 5.86\text{B}$ dense parameters:

$$b(\Pi, T) = \underbrace{\frac{6}{\rho} \left(\frac{P_e}{\eta} + P_d \right)}_{\text{weights+gradients}} + \underbrace{\frac{12 P_e}{G} + \frac{12 P_d}{\rho \delta}}_{\text{optimizer state}} + \underbrace{\alpha \frac{T}{\kappa} + \frac{4 |\mathcal{V}| T}{\kappa} \mathbf{1}[\text{terminal stage}]}_{\text{activations+output logits}} + b_0 \leq M, \quad (21)$$

with η the expert-sharding degree, T the token wall, $|\mathcal{V}| = 248,320$, a measured per-token activation coefficient α , and a small constant allocator residue b_0 amounting to a few percent of M . Equation 21 reproduced the observed abort to within a fraction of a percent of M , and its structure rather than its numeric value determines the plan.

Proposition 7.1 (Expert optimizer state is plan-invariant). The optimizer footprint of the expert parameters depends only on G , never on how Π distributes them.

Per-device expert parameters shrink as $1/\eta$, but the distributed-optimizer shard count grows as $\tau\kappa\delta/\eta$; their product is exactly G , so the two effects cancel. The consequence was the most expensive lesson of bring-up: *reducing* expert sharding to relieve memory pressure doubles resident weights and gradients while leaving optimizer state untouched, so η is pinned maximal. Tensor parallelism beyond $\tau = 2$ is likewise inert, since the recurrent blocks are not tensor-sharded and the sequence is re-gathered to full length before them, so both their parameters and their activation peak replicate rather than divide. Pipeline and context degrees are the only real levers, and the binding term at long horizons is the fixed-point output projection on the terminal stage: at an 84k wall, $\kappa = 2$ leaves a logit buffer occupying roughly a third of M and therefore exceeds it, whereas $\kappa = 4$ halves that share and admits the plan.

Key Insight: capacity scales out, not in

Proposition 7.1 says the dominant memory term of a sparse actor–critic pair is invariant to every redistribution of experts and can be reduced *only* by enlarging G . Feasibility is therefore a property of the device budget rather than of a tuning search, and this is what makes the recipe portable across device budgets spanning more than a factor of two: a plan is admitted analytically, before the 15 to 20 min initialization, instead of empirically after an hour-long step dies.

7.2 Context parallelism for the recurrent operator

Equation 21 divides activations by κ , but the generic context-parallel path is unsound for a recurrent scan, which had historically forced $\kappa = 1$ and made the single longest trajectory the memory wall. We therefore sequence-shard the recurrent operator natively: each rank evaluates its own token segment and hands the carried state to its successor, so the scan remains sequentially exact while its activation residency divides. One subtlety is load-bearing: the trainer distributes context shards in an interleaved layout for gradient balance, whereas state passing requires contiguity, so shards are re-laid out and the packed-sequence descriptor records how many interleaved segments a sample contributes, a distinction that is unrecoverable from cumulative offsets alone, since real and padding segments can present identical divisibility.

Correctness is asserted rather than assumed: against a single-rank reference the forward pass is bit-exact and gradients agree to 1.2×10^{-7} at $T = 33,792$ and $65,536$. Relative to $\kappa = 1$, recurrent activation peaks fall to roughly one half, one quarter and one eighth at $\kappa \in \{2, 4, 8\}$, a near-linear reduction and precisely what makes 84k and 128k walls admissible on an unchanged device budget. Two costs are accepted knowingly. Since $\tau\rho\kappa\delta = G$ is fixed, raising κ consumes data parallelism: a batch of 512 at $\delta = 2$ becomes 256 serially accumulated micro-batches per rank. Second, the kernel’s long-sequence algorithm selection must be pinned, since the library silently switches implementations under inference-mode heuristics, decoupling trainer numerics from the sampler and reintroducing the very engine gap that Section 4 exists to close.

7.3 Liveness under hour-scale steps

At our production device budget, where a single step takes approximately one hour, events with per-hour probability 10^{-2} are per-run certainties, and the failures we observed were not crashes but *indefinite waits*: a single unresponsive participant stalls a publication barrier that spans the entire training partition, and the job dies of a downstream timeout whose message names the barrier rather than the cause. Two principles proved sufficient. First, every control-plane operation carries a bounded deadline, while bulk transfers deliberately do not, since a multi-minute collective copy is legitimate whereas an unbounded teardown request is a liveness hole. Second, detection must distinguish *death* from *slowness*: deadlines inherited from single-node defaults misclassify benign stragglers as failures, since a first-step kernel compilation or a cold-cache rank is slow by construction. Probes therefore carry a generous initial grace of 600s before steady-state checks at a 30s period and a 120s deadline, and re-created collective groups take deadlines calibrated to the regime of 30 to 120min rather than to interactive latencies. Unresponsive replicas are reclaimed at the next publication

barrier, amortizing recovery into a synchronization point that already exists. Checkpointing is deliberately asymmetric, saving weights only and every ten steps, which trades resumability for cost on the understanding that a crash means restart rather than resume.

7.4 Environment concurrency and the straggler tail

Reward evaluation is an external service, so the rollout layer is an admission-control problem. Environment instances are hosted by per-node auxiliary workers pinned to the inference partition with a per-node concurrency cap (35 at batch 256, 70–94 at batch 512), keeping trial traffic off the coordinator, and creation is paced by a token bucket at 6 s^{-1} against a provider quota of 600 min^{-1} . Instances are provisioned minimally, with a single core and a small memory and disk allotment, because concurrency, not per-instance capability, sets the achievable batch; reclamation must be explicit, since deferred deletion that triggers only after suspension leaks instances until quota exhaustion. Per-trial deadlines are layered (5400 s remote, 3600 s agent, 900 s verifier).

The scheduler oversamples: it launches 560 trials for a batch of 512, admits the first 512 to complete, and cancels the remainder. This bounds step time against a heavy-tailed completion-time distribution, but the residual cost is structural. Once a handful of trials remain, no admission decision can be made until one terminates, and near-deadline trajectories are exactly those grinding through context compaction at degraded decode rates; one episode spent 17 minutes at 596 trajectories collected against 512 groups required. The cancelled tail is not a uniform sample but concentrates the hardest task families, so oversampling trades wall-clock determinism against a selection bias quantified in Section 10.

7.5 The publication barrier and rollout/training balance

Updated parameters are published to every inference replica once per step over dedicated collectives, with generation quiesced first so that no request straddles two versions; this is the barrier that makes the staleness of Appendix B exactly one update rather than an unmodeled random variable.

Because the two partitions are disjoint and pipelined one step deep, step time is max rather than sum, which turns device allocation into a genuine optimization. Let n devices serve inference and $G - n$ train; with per-replica throughput approximately additive,

$$T_{\text{step}}(n) = \max\left(\underbrace{\frac{R}{n}}_{\text{rollout}}, \underbrace{\frac{C}{G-n}}_{\text{training}}\right), \quad n^* = \arg \min_n T_{\text{step}}(n) \text{ attained at equality.} \quad (22)$$

Measurement confirms the shape: in an earlier configuration, moving from one to three inference replicas cut generation from 58.9 to 19.6 min (single-replica rate 4.35 groups/min), reducing a step of approximately 60 min to approximately 23 min, a factor of 2.6 with no additional hardware, purely by rebalancing n . The production plan sits near the balance point and spends about half of the device budget on inference, hiding approximately 16 min of generation under approximately 41 min of training compute at $\delta = 2$, for a step of approximately 57 min. That the terms are deliberately *unequal* is the caveat: generation slack absorbs the straggler tail of Section 7.4 without exposing it in T_{step} .

Algorithm 3. One outer iteration. Generation for step $t+1$ overlaps the update at step t , and the publication barrier is the only synchronization point between the two partitions, which is what pins the behaviour policy to a single version.

Require: plan Π ; batch B ; oversampling factor $1+\varsigma$

- 1: **assert** $b(\Pi, T) \leq M$ and $\tau\rho\kappa\delta = G$ $\triangleright$ fail fast, before initialization
- 2: **for** $t = 0, 1, \dots$ **do**
- 3: **inference partition:** admit $\lceil (1+\varsigma)B \rceil$ trials under the rate bound; collect the first B to terminate; cancel the tail
- 4: **training partition:** update the critic on $\mathcal{B}_t$, then the actor against its pre-update values (Section 4.5)
- 5: quiesce generation; publish θ_{t+1} ; reclaim dead replicas; rotate the behaviour snapshot $\triangleright$ single barrier
- 6: **end for**

8 Related Work

Data synthesis and training for terminal agents. Expert-authored terminal tasks pair an instruction with a container and an executable verifier [14], but manual authoring does not reach training scale, so recent work synthesizes environments along three routes. *Repository-derived* methods recover workspaces from real development histories and reuse the accompanying tests [7, 15, 27]; *perturbation* methods inject faults into working repositories or CLI workspaces [10, 29]; and *task-conditioned synthesis* generates the instruction, environment and verifier jointly from categories, capability taxonomies or skill graphs [1, 3, 5, 6, 16, 35]. Because tasks synthesized from scratch saturate quickly against frontier models, a second line makes the pool itself adaptive: RST recursively re-seeds validated task bundles to lengthen horizons [8], SETA and environment evolution raise difficulty generation by generation [2, 20], CalibForge calibrates against solver feedback [13], and Terminal-Universe reconstructs executable workspaces from recorded trajectories before re-querying them across workspaces and dialogue rounds [26]. On the training side, most of these pipelines are validated by supervised fine-tuning alone, and the reinforcement-learning evidence is confined to comparatively small dense policies optimized against binary outcomes, with reported gains over the SFT checkpoint often within a few points [3, 6, 18, 25].

Stable reinforcement learning under training–inference mismatch. Modern RL stacks sample with an inference engine and differentiate with a training engine [21, 23, 34], so differing kernels, numerics and parallelism make the sampler’s log-probabilities diverge from the trainer’s even at zero staleness, corrupting the importance ratios rather than merely dating them [11]. Sparse models amplify this, since one update flips roughly a tenth of the activated experts for the same prefix and token ratios then compare two different subnetworks [33]. Existing remedies reweight, mask, or reshape the objective. Truncated importance sampling corrects the ratio in place [32], though token-level correction leaves the induced state distribution biased [11]; IcePop instead drops gradients outside a fixed two-sided band [31], KPop replaces that band with a binary-KL acceptance region so exploratory low-probability tokens are not over-masked [4], and SAT contracts only the sign-selected clip endpoint on a self-calibrating staleness quantile [30]; GSPO and DPPO act on the objective itself, moving the ratio to the sequence level or replacing ratio clipping with a direct divergence estimate [17, 33]. Closest to us are methods that remove the mismatch at its source: rollout routing replay reinstates the sampler’s expert selections in the backward pass [12], and token-in-token-out makes the trainer score exactly the identifiers the engine consumed and emitted [22].

9 Lessons learned and what did not work

The failures that most changed our understanding of long-horizon RL concerned three parts of the learning problem: what information the reward provides, what history the agent can use, and which tasks reach the optimizer. Each can change the training outcome while leaving the PPO update itself intact.

9.1 Critic calibration does not guarantee policy improvement

In the 27B pathfinding runs, increasing the critic learning rate brought the first positive explained variance forward from step 50 to step 30, yet rollout reward remained flat across the compared settings (Section 4.5). The critic was learning to predict returns more effectively without a corresponding improvement in the reward earned by the actor. This redirected attention to the information supplied by the tasks and their verifiers.

The critic learns to predict the return that the reward function assigns. When hard tasks mostly receive the same zero outcome, improving that prediction cannot reveal partial progress that the verifier never rewards. The task pool and reward granularity determine which improvements are observable; critic calibration determines how well the value baseline models the resulting returns. Our dense reward addresses the former by making partial completion visible, while critic warm-starting and faster value learning address the latter.

The practical lesson is to diagnose these requirements separately. Explained variance is useful evidence about the value function, while passing tests, task completion, and held-out evaluation establish whether the policy is improving. When calibration gets better but reward does not move, task difficulty and verification granularity deserve attention alongside further optimizer tuning.

9.2 Context management determines what the agent can learn

An agent can become less efficient because it loses track of what it has already done. In one campaign, a mismatch between the context budget and the requested summary length caused 98.3% of full-summary attempts to fail. The fallback retained only a short fragment of the history. Agents repeated completed work, average turns rose from 22 to 30, and more trajectories reached their time limits. The run continued to produce trajectories and rewards, but the agent was making decisions with degraded memory.

For a long-horizon agent, context compaction determines which earlier observations and decisions remain available to the policy. Losing that information changes the effective task presented at subsequent turns. Training can then spend capacity learning to cope with avoidable information loss. Faithfully optimizing the collected trajectory does not restore the history that was missing when its actions were chosen.

This makes turn growth alone an ambiguous diagnostic: it can reflect useful additional work, reward-driven repetition, or a failure to preserve context. Before changing the reward in response, we need to inspect what the agent remembers across compaction boundaries and whether it repeats work whose results were lost. Applying a length penalty to such trajectories would leave the memory failure unresolved.

9.3 Rollout throughput changes the training distribution

We deliberately mix task difficulties through shuffling, but the rollout collection rule introduces another selection step. To bound batch latency, the scheduler launches more trials than needed, accepts the first completed batch, and cancels the remaining tail (Section 7.4). In one measured step, 48 of 561 submitted trials were cancelled even though the accepted batch was full. A completed-batch counter therefore concealed the loss of those training opportunities.

Completion time varies with task family, difficulty, and the agent’s behaviour, so selection by completion time can favour faster tasks. Shuffling the launch order does not remove this bias: the distribution that reaches the optimizer also depends on which trajectories survive collection. This is especially consequential for our dense reward design. Partial progress on hard tasks is useful only if those trajectories are retained long enough to be verified and used in an update. A richer reward cannot recover supervision from a cancelled trajectory.

Throughput must therefore be assessed together with task retention, including which families and trajectory lengths are being dropped. We retain oversampling for its wall-clock benefit, while treating its effect on long-task coverage as an open trade-off. Preserving unfinished trajectories through partial-rollout continuation is one direction discussed in Section 10; its benefit to final task performance remains to be established.

9.4 Why PPO rather than a critic-free group baseline

Before committing to PPO with a full-size critic, we trained GRPO on T1-15k under the same harness, reward and device budget. The rollout reward in Figure 16 fluctuates without a trend, and the held-out evaluation is identical at the two checkpoints we scored, **51.7%** at both step 10 and step 20, which is the same 46 of 89 tasks resolved. Three properties of the group-relative estimator account for this, and each is specific to long horizons.

The advantage degenerates precisely where signal is required. GRPO replaces the value baseline with the within-group statistics of G trajectories drawn from one task, $A_i = (r_i - \mu)/\sigma$, so a group whose members all fail, or all succeed, contributes no gradient. On tasks hard enough to motivate the dense reward of Section 5, such groups are the common case, and the few that do vary dominate the update. A single advantage is then shared by every token of a trajectory, which removes the temporal credit assignment that GAE supplies.

Group size and task diversity compete for long-horizon tasks. Each task must be sampled G times before it yields one advantage, so at fixed generation capacity the number of distinct tasks per step falls by that factor, and the effective batch narrows to a small set of repeatedly sampled tasks. PPO spends the same budget on distinct tasks, because a learned value function provides the baseline that GRPO has to purchase with repeated long-horizon rollouts.

The cost of those repeats is paid at the tail. A group closes only when its slowest member terminates, and terminal trajectories are heavy-tailed in both turns and tokens, so the truncation that bounds step time for independent trials cannot be applied inside a group without discarding the comparison it exists to form. Generation therefore idles at the very configuration the estimator requires. Taken together, the group baseline is a substitute for a value function whose cost is modest in single-turn settings and grows with the horizon, whereas the critic’s cost is real but bounded, as Section 4.5 sets out.



Figure 16 Rollout reward of the GRPO campaign on T1-15k under the same harness and reward as the production run. The faint line is the per-step mean and the bold line an exponential moving average at $\alpha=0.25$.

10 Limitations and future work

Conditioning is exact only up to re-tokenized history. TITO guarantees exactness on the loss region, but in the re-tokenized case later turns are trained conditioned on sampled identifiers where inference conditioned on the harness’s re-tokenized ones, which affects 2.6% of tokens in the audited run. Eliminating the residual would require the harness to carry token identifiers as the source of truth for history, which we did not attempt.

No single-axis reward ablation yet. The evidence comparing binary against dense rewards is campaign-level, since the two runs differ in reward, pool, initialization and routing replay at once. A binary run matched on pool, initialization and routing replay would isolate the reward’s contribution.

Verifier integrity is filtered, not enforced. The verifier executes in the agent-controlled sandbox with no runtime tamper detection; defense currently rests on the audited task pool. In-sandbox integrity checks (read-only test mounts, checksummed verifiers) are straightforward next steps.

The long tail is being paid for twice. Skim-oversampling drops the hardest trials from training, and the case study shows remaining evaluation failures concentrate on exactly such tasks (hundreds of turns, timeouts).

Partial-rollout continuation, per-task budget respect, larger evaluation budgets, and difficulty-aware scheduling are all in flight.

Data distribution. Failures concentrate in machine learning / data-science / scientific-computing categories that T1-15k under-covers. In the future, we hope to explore more comprehensive data distributions to enable trained models to achieve versatility across various domain categories.

Staleness and asynchrony. The production loop is one-step asynchronous; the fully-asynchronous path exists in the plugin but was not used for the 122B campaigns. Studies [30] on Qwen3-30B-A3B (AIME24) show baseline GRPO/GSPO collapsing under staleness 8 and identify combinations (GSPO+SAT+R³, soft masking, TIS) that survive; porting the winning combination to the terminal-agent setting is planned. The meeting record also discusses critic *value pretraining* on offline trajectories and classification-based value losses (HL-Gauss) for step-0 explained variance; neither is implemented in this codebase today.

Reporting gaps. Several bookkeeping items remain outstanding: the final optimizer-placement setting of the production runs, the coverage of the length-adaptive advantage variant, the per-row evaluation protocol labels, and the weight-synchronization timings. These are bookkeeping matters rather than blockers.

Scope. Everything here is one model family, one agent harness and one benchmark. The transferable claims are the mechanisms and their failure modes, namely token-faithful trajectory construction, routing replay, critic scheduling and the operational guardrails, rather than the specific numbers.

References

- [1] Zhiyuan Fan, Tinghao Yu, Yuanjun Cai, Jiangtao Guan, Yun Yang, Dingxin Hu, Jiang Zhou, Xing Wu, Zhuo Han, Feng Zhang, and Lilin Wang. Toward scalable terminal task synthesis via skill graphs, 2026. <https://arxiv.org/abs/2604.25727>.
- [2] Zhiyuan Fan, Tinghao Yu, Yuanjun Cai, Jiang Zhou, Jiangtao Guan, Jincheng Liu, Yun Yang, Dingxin Hu, Zhuo Han, Xing Wu, Feng Zhang, and Lilin Wang. Environment evolution for terminal agents, 2026. <https://arxiv.org/abs/2609.04128>.
- [3] Kanishk Gandhi, Shivam Garg, Noah D. Goodman, and Dimitris Papailiopoulos. Endless terminals: Scaling RL environments for terminal agents, 2026. <https://arxiv.org/abs/2601.16443>.
- [4] Jia Guo, Yan Sun, Zhenyu Huang, Zihao Wang, Zujie Wen, Zhiqiang Zhang, Jun Zhou, and Stanley Kok. KPop: Taming training–inference mismatch in reinforcement learning with adaptive masking regions. <https://ringtech.notion.site/kpop>, May 2026.
- [5] Zhanbo Hua, Yifan Yao, Weihao Xie, Yongchi Zhao, Minghao Liu, Ruizhi Qiu, Zhewei Huang, Zun Wang, Yiyan Ji, Yunhai Ye, Letian Zhu, Xinping Lei, Han Li, Zhiyuan Ma, Zili Wang, Zhaoxiang Zhang, and Jiaheng Liu. CLI-Universe: Towards verifiable task synthesis engine for terminal agents, 2026. <https://arxiv.org/abs/2606.22883>.
- [6] Hamish Ivison, Junjie Oscar Yin, Rulin Shao, Teng Xiao, Nathan Lambert, and Hannaneh Hajishirzi. Tmax: A simple recipe for terminal agents, 2026. <https://arxiv.org/abs/2606.23321>.
- [7] Naman Jain, Jaskirat Singh, Manish Shetty, Liang Zheng, Koushik Sen, and Ion Stoica. R2E-Gym: Procedural environments and hybrid verifiers for scaling open-weights SWE agents, 2025. <https://arxiv.org/abs/2504.07164>.
- [8] Zhongzhi Li, Yucheng Shi, Zongxia Li, Ruhan Wang, Anhao Li, Zixun Huang, Junyao Yang, Lei Ke, Ninghao Liu, Haitao Mi, and Leowei Liang. Recursive synthesis for long-horizon terminal tasks, 2026. <https://arxiv.org/abs/2608.05466>.
- [9] Zongxia Li, Zhongzhi Li, Yucheng Shi, Ruhan Wang, Junyao Yang, Zhichao Liu, Xiyang Wu, Anhao Li, Yue Yu, Ninghao Liu, Lichao Sun, Haotao Mi, and Leowei Liang. Long-horizon-terminal-bench: Testing the limits of agents on long-horizon terminal tasks with dense reward-based grading, 2026. <https://arxiv.org/abs/2607.08964>.

- [10] Yusong Lin, Haiyang Wang, Shuzhe Wu, Lue Fan, Feiyang Pan, Sanyuan Zhao, and Dandan Tu. CLI-Gym: Scalable CLI task generation via agentic environment inversion, 2026. <https://arxiv.org/abs/2602.10999>.
- [11] Jiakai Liu, Yingru Li, Yuqian Fu, Jiawei Wang, Qian Liu, and Yu Shen. When speed kills stability: Demystifying RL collapse from the training–inference mismatch, Sep 2025. <https://richardli.xyz/rl-collapse>.
- [12] Wenhan Ma, Hailin Zhang, Liang Zhao, Yujiao Song, Yiyuan Wang, Zhifang Sui, and Fuli Luo. Stabilizing MoE reinforcement learning by aligning training and inference routers. *arXiv preprint arXiv:2510.11370*, 2025.
- [13] Fanzhe Meng, Guoxin Chen, Jiale Zhao, Shuang Sun, Zhiyu Lin, Wayne Xin Zhao, Ruihua Song, Ji-Rong Wen, and Kai Jia. CalibForge: Adversarial solver calibration for scaling learnable terminal tasks, 2026. <https://arxiv.org/abs/2608.06352>.
- [14] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. <https://arxiv.org/abs/2601.11868>.
- [15] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with SWE-Gym, 2024. <https://arxiv.org/abs/2412.21139>.
- [16] Renjie Pi, Grace Lam, Mohammad Shoeybi, Pooya Jannaty, Bryan Catanzaro, and Wei Ping. On data engineering for scaling LLM terminal capabilities, 2026. <https://arxiv.org/abs/2602.21193>.
- [17] Penghui Qi, Xiangxin Zhou, Zichen Liu, Tianyu Pang, Chao Du, Min Lin, and Wee Sun Lee. Rethinking the trust region in LLM reinforcement learning. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. arXiv:2602.04879.
- [18] Negin Raoof, Richard Zhuang, Marianna Nezhurina, Etash Guha, Atula Tejaswi, et al. OpenThoughts-Agent: Data recipes for agentic models, 2026. <https://arxiv.org/abs/2606.24855>.
- [19] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [20] Qijia Shen, Zhiqi Huang, Vamsidhar Kamanuru, Aznaur Aliev, Jay Rainton, Ahmed Awelkair, Zhichen Zeng, Jiajun Li, Shi Dong, Yueming Yuan, Boyuan Ma, Qizheng Zhang, Jiwei Fu, Yuzhen Mao, Wendong Fan, Ping Nie, Philip Torr, Bernard Ghanem, Changran Hu, Jonathan Lingjie Li, Urmish Thakker, and Guohao Li. SETA: Scaling environments for terminal agents, 2026. <https://arxiv.org/abs/2607.10891>.
- [21] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [22] The Miles Team and LMSYS Org. No token left behind: Demystifying token-in-token-out in miles. <https://www.lmsys.org/blog/2026-05-13-no-token-left-behind>, 2026. Blog post on the token-in-token-out principle in the Miles RL framework.
- [23] THUDM and the slime contributors. slime: an llm post-training framework for rl scaling. <https://github.com/THUDM/slime>, 2026. v0.3.0; this work builds on commit bf14dc21.
- [24] Ruhan Wang, Yucheng Shi, Zongxia Li, Zhongzhi Li, Yue Yu, Junyao Yang, Kishan Panaganti, Haitao Mi, Dongruo Zhou, and Leoweiliang. Harness handbook: Making evolving agent harnesses readable, navigable, and editable, 2026. <https://arxiv.org/abs/2607.13285>.

- [25] Weixun Wang, XiaoXiao Xu, Wanhe An, Fangwen Dai, Wei Gao, Yancheng He, et al. Let it flow: Agentic crafting on rock and roll, building the ROME model within an open agentic learning ecosystem, 2026. <https://arxiv.org/abs/2512.24873>.
- [26] Jie Wu, Zhenru Zhang, Beichen Zhang, Xuwu Wang, Yuhui Su, Mouxiang Chen, Peng Wang, Zhihai Wang, Que Shen, Hao Zhou, An Yang, Fei Huang, Yujiu Yang, and Dayiheng Liu. Terminal-Universe: Turning agent trajectories into scalable terminal environments, 2026. <https://arxiv.org/abs/2609.04148>.
- [27] Siwei Wu, Yizhi Li, Yuyang Song, Wei Zhang, Yang Wang, Riza Batista-Navarro, Xian Yang, Mingjie Tang, Bryan Dai, Jian Yang, and Chenghua Lin. Large-scale terminal agentic trajectory generation from dockerized environments, 2026. <https://arxiv.org/abs/2602.01244>.
- [28] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [29] John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. SWE-smith: Scaling data for software engineering agents. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2025. <https://arxiv.org/abs/2504.21798>.
- [30] Junyao Yang, Yucheng Shi, Zongxia Li, Zhongzhi Li, Ruhan Wang, Xiangxin Zhou, Kishan Panaganti, Haitao Mi, and Leowei Liang. Stale but stable: Staleness-adaptive trust regions for stabilizing asynchronous reinforcement learning, 2026. <https://arxiv.org/abs/2607.18722>.
- [31] Xin Zhao, Yongkang Liu, Kuan Xu, Jia Guo, Zihao Wang, Yan Sun, Xinyu Kong, Qianggang Cao, Liang Jiang, Zujie Wen, Zhiqiang Zhang, and Jun Zhou. Small leak can sink a great ship—boost RL training on MoE with IcePop! <https://ringtech.notion.site/icepop>, Sep 2025.
- [32] Chujie Zheng, Kai Dang, Bowen Yu, Mingze Li, Huiqiang Jiang, Junrong Lin, Yuqiong Liu, Hao Lin, Chencan Wu, Feng Hu, An Yang, Jingren Zhou, and Junyang Lin. Stabilizing reinforcement learning with LLMs: Formulation and practices, 2025. <https://arxiv.org/abs/2512.01374>.
- [33] Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, Jingren Zhou, and Junyang Lin. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.
- [34] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. *arXiv preprint arXiv:2312.07104*, 2023.
- [35] Kaijie Zhu, Yuzhou Nie, Yijiang Li, Yiming Huang, Jialian Wu, Jiang Liu, Ximeng Sun, Zhenfei Yin, Lun Wang, Zicheng Liu, Emad Barsoum, William Yang Wang, and Wenbo Guo. TermiGen: High-fidelity environment and robust trajectory synthesis for terminal agents, 2026. <https://arxiv.org/abs/2602.07274>.

Appendix

A Notation

Tables 2–5 collect every symbol used in the paper. Sections 4.1–4.4 use exactly these conventions. Three of them are worth stating explicitly up front, because they carry most of the argument of Section 4.

1. **Policy versions are indexed by the training step at which the weights were produced.** π_t is the policy whose parameters are θ_t , *i.e.* the weights the trainer holds while performing update t ; π_{t-1} is the immediately preceding version. In the asynchronous loop the batch consumed at step t was *generated* under π_{t-1} , so the two symbols are never interchangeable (Appendix B).
2. **Superscripts r and t mark the *engine*, not the version.** π^r denotes evaluation by SGLang (rollout) and π^t evaluation by Megatron (trainer). Engine and version are orthogonal axes: π_{t-1}^r and π_{t-1}^t are the *same* weights under two implementations, which is precisely the discrepancy TITO and R³ address.
3. **Bold lowercase is a token-indexed sequence; calligraphic uppercase is a set or an index collection.** Thus $\mathbf{a}_i$ is turn i ’s completion and $\mathbb{I}_j^\ell$ is the expert index set chosen at layer ℓ for position j .

Two pairs of symbols are deliberately close and must not be conflated: the PPO clip ε versus the router-perturbation bound ϵ , and the discount γ versus the top- k margin γ_j^ℓ . Both distinctions are flagged in the tables.

Table 2 Index conventions. These are fixed throughout the paper; in particular t is never a token index.

Symbol	Type	Meaning
t	integer	<i>Training step</i> / policy version. Never a token index.
j	integer	<i>Position</i> in the stitched token stream, $j \in [N]$.
i	integer	<i>Turn</i> within a trial, $i \in [T]$.
ℓ, e	integers	MoE layer $\ell \in [L]$ and expert $e \in [E]$.
n	integer	Lookahead offset inside the GAE sum (Equation 16).

Table 3 Policy versions, engines, and optimization state (Appendix B).

Symbol	Type	Meaning
θ_t, π_t	params, policy	Actor parameters after update t , and the induced policy. π_0 is the SFT initialization.
π_{t-1}	policy	The previous version; in the asynchronous loop, the version that <i>generated</i> the batch consumed at step t .
$\mathcal{B}_t$	set	The batch consumed by update t ; $\mathcal{B}_t \sim \pi_{t-\sigma}^r$ (Equation 23).
π^r, π^t	policy	Same weights evaluated by the sampler versus by the trainer. Superscripts compose with version subscripts: π_{t-1}^r .
π_{old}	policy	Behaviour policy of the PPO ratio. Here it is π_{t-1}^t , the previous version recomputed on the training side rather than read back from the sampler (Equation 26).
$\pi_{\text{ref}}, \theta_{\text{ref}}$	policy	Frozen reference for the KL diagnostic; routes with its own TopK _{k} .
σ	integer	Staleness, $\sigma = t -$ (version that generated $\mathcal{B}_t$). Production runs use $\sigma = 1$.
ϕ_t, V_ϕ	params, function	Critic parameters after update t , and the value function. $V_{\text{old}} = V_{\phi_{t-1}}$ (Equation 16).
η_θ, η_ϕ	scalars	Actor and critic learning rates; $\eta_\phi = 30 \eta^{\text{canon}}$, $\eta_\theta = 2 \eta^{\text{canon}}$ with $\eta^{\text{canon}} = 5 \times 10^{-7}$ (Section 4.5).

Table 4 Trajectories, tokens, and MoE routing (Sections 4.2, 4.3).

Symbol	Type	Meaning
$\mathcal{V}$	set	Vocabulary.
enc, dec	maps	Tokenizer and its inverse rendering. Token drift is exactly the failure of $\text{enc} \circ \text{dec} = \text{id}$.
T	integer	Number of turns in a trial.
$\mathcal{M}_i$	sequence	Message history presented to the template at turn i .
$\mathbf{p}_i, \mathbf{a}_i$	$\in \mathcal{V}^*$	Turn i 's prompt token ids and sampled completion token ids.
$\hat{\mathbf{a}}_i$	$\in \mathcal{V}^*$	The harness's re-encoding $\text{enc}(\text{dec}(\mathbf{a}_i))$; never enters the training stream (Equation 10).
$\mathbf{q}_i$	$\in \mathbb{R}^*$	Sampler log-probabilities for $\mathbf{a}_i$ (per turn).
$\mathbf{x}, N$	$\in \mathcal{V}^N$, integer	The stitched training token stream and its length; x_j is its j -th token.
$\mathbf{m}, m_j$	$\in \{0, 1\}^N$	Loss mask; $m_j = 1$ iff x_j was sampled by π_{t-1}^r (Equation 5).
$\mathbf{q}, q_j$	$\in \mathbb{R}^N$	The per-turn $\mathbf{q}_i$ stitched onto the stream, 0 where $m_j = 0$.
$\mathbf{u}, \mathbf{v}$	$\in \mathcal{V}^*$	Generic token sequences, used only to state the prefix relations.
$\ , \preceq, \ominus$	operators	Concatenation; "is a prefix of"; residual suffix after removing a prefix.
drop_s	operator	Removal of s tokens from a sequence edge (Equation 7).
$\mathbf{p}_{i+1}[b:e]$	slice	Half-open token span located by the generation-time offset map.
$\mathcal{A}, \mathcal{D}, \mathcal{P}$	sets	Partition of the N stream positions into exactly-aligned, re-tokenized and placeholder positions (Equation 11).
L, k, E	integers	MoE layers, top- k width, experts per layer; (48, 8, 256) for Qwen3.5-122B-A10B.
d	integer	Model hidden size.
$\mathbf{h}_j^\ell, \mathbf{y}_j^\ell$	$\in \mathbb{R}^d$	Layer- ℓ input and output at position j .
$\mathbf{s}_j^\ell$	$\in \mathbb{R}^E$	Router scores; $[\mathbf{s}_j^\ell]_e$ is expert e 's score.
f_e	map	Expert e 's feed-forward transform.
$\mathbb{I}_j^\ell$	$\subset [E]$	Selected expert set, $ \mathbb{I}_j^\ell = k$ (Equation 2).
$\mathbb{I}_j$	tuple	$(\mathbb{I}_j^1, \dots, \mathbb{I}_j^L)$: the effective sub-network at position j .
$g_{j,e}^\ell$	scalar	Gate weight applied to expert e ; gradient flows through it.
γ_j^ℓ	scalar	Top- k margin $[\mathbf{s}_j^\ell]_{(k)} - [\mathbf{s}_j^\ell]_{(k+1)}$. Distinct from the discount γ .
ξ_j^ℓ, ϵ	vector, scalar	Cross-engine router-score perturbation and its bound $\ \xi_j^\ell\ _\infty \leq \epsilon$. Distinct from the GAE residual δ_j and the PPO clip ϵ .
$\mathbf{R}$	$\in [E]^{\rho \times L \times k}$	Recorded routing tensor; $\mathbf{R}[j, \ell, :]$ is the expert set used to predict position $j+1$ (Equation 13). $\mathbf{R}_i$ is turn i 's capture.
ρ_i, ρ	integers	Routing rows per turn, $\rho_i = \mathbf{p}_i + \mathbf{a}_i - 1$, and for the stitched stream, $\rho = N - 1$ (a turn's last token predicts nothing).
Π	operator	Sharding composition applied identically to $\mathbf{x}$ and $\mathbf{R}$: $\Pi = \Pi_{\text{SP}} \circ \text{pad}_{\text{TP} \times \nu} \circ \Pi_{\text{CP}} \circ \text{pad}_1$, with Π_{CP} the interleaved context-parallel slice and Π_{SP} the local sequence-parallel shard (Appendix B.3).
ν	integer	Padding granularity of the packed sequence.
$\mathcal{S}$	mode	Router mode: free selection, replayed selection without gradient, or replayed selection with gradient.

Table 5 RL objective and diagnostics (Sections 4.4, 4.5).

Symbol	Type	Meaning
s_j, a_j, x_j	state, action, token	Conditioning prefix, emitted token, and stream token at position j .
$r_j(\theta)$	scalar	PPO importance ratio (Equation 1). Distinct from the per-position reward $\hat{r}_j$.
$\hat{r}_j$	scalar	Reward credited at position j ; nonzero only on a trajectory's final token (Section 5.2).
ϵ	scalar	PPO clip ratio, 0.2. Distinct from the router-perturbation bound ϵ .
$\hat{A}_j, \hat{R}_j, \delta_j$	scalars	GAE advantage, return, and TD residual (Equation 16).
γ, λ	scalars	Discount and GAE decay; both 1.0 here. γ is distinct from the margin γ_j^ℓ .
$\mathcal{L}^{\text{PPO}}, \mathcal{L}^V$	scalars	Policy and value objectives.
EV	scalar	Critic explained variance (Equation 19).
Δ_j	scalar	Per-token train-inference log-probability discrepancy (Equation 24).
$ \Delta \log p $	scalar	Mask-weighted mean $ \Delta_j $ over the loss region (Equation 15).
S	integer	Number of unit tests in a task's verifier (Section 5).

B Asynchrony, version skew, and routing alignment

This appendix makes precise what one-step asynchrony means for the quantities of Section 4, and why routing alignment must be defined against a specific version rather than against the policy in the abstract.

B.1 The one-step-ahead loop

Training and inference occupy disjoint devices, and the rollout for step $t+1$ generates while step t trains. Writing $\mathcal{B}_t$ for the batch consumed by update t , the loop maintains

$$\mathcal{B}_t \sim \pi_{t-\sigma}^r, \quad \theta_t = \theta_{t-1} - \eta_\theta \nabla_\theta \mathcal{L}^{\text{PPO}}(\theta_{t-1}; \mathcal{B}_t), \quad \sigma = 1, \quad (23)$$

with the publication of θ_t synchronized against in-flight generation so that no request ever spans two versions. The staleness σ is therefore a constant of the production configuration rather than a random variable, since every sample in $\mathcal{B}_t$ was drawn under exactly π_{t-1} . A fully asynchronous path with unbounded σ exists but was not used for the 122B campaigns.

B.2 Three distinct gaps, one measured quantity

Because version and implementation are independent axes, the discrepancy Δ_j that Section 4.4 measures decomposes into terms with different causes and different remedies. For a loss-bearing position j ,

$$\Delta_j = \underbrace{\log \pi_t^t(x_j | \cdot) - \log \pi_{t-1}^t(x_j | \cdot)}_{\text{(a) version skew, genuine learning}} + \underbrace{\log \pi_{t-1}^t(x_j | \cdot) - \log \pi_{t-1}^r(x_j | \cdot)}_{\text{(b) implementation gap, to be eliminated}}, \quad (24)$$

$$\text{(b)} = \underbrace{[\text{token drift}]}_{\text{TITO, Equation 3}} + \underbrace{[\text{routing divergence}]}_{R^3, \text{Equation 4}} + \underbrace{[\text{kernel numerics}]}_{\text{irreducible here}}. \quad (25)$$

Three consequences follow, and they explain the shape of every mismatch curve in this report.

Only the second term is a defect. Term (a) is the distance the policy legitimately travelled during one update. It is what PPO’s clip exists to bound, and it should be nonzero. Term (b) is bookkeeping error, namely the same weights θ_{t-1} disagreeing with themselves across two implementations, and TITO together with R^3 target it exclusively. This is why the slow upward drift of $|\Delta \log p|$ over a long run is not a regression, as Section 4.4 notes: as the policy improves, term (a) grows while term (b) stays flat.

The behaviour policy is evaluated on the training side by choice. The PPO denominator is recomputed by the trainer rather than read back from the sampler, so Equation 1 instantiates as

$$r_j(\theta) = \frac{\pi_t^t(x_j | x_{<j}, \mathbb{I}_j)}{\pi_{t-1}^t(x_j | x_{<j}, \mathbb{I}_j)}, \quad \mathbb{I}_j = \mathbf{R}[j, :, :] \text{ for both numerator and denominator.} \quad (26)$$

Both factors are evaluated by the same implementation on the same routing, so term (b) cancels from the ratio provided $\mathbb{I}_j$ is held fixed across the two passes, which is exactly what R^3 guarantees and what Algorithm 1 schedules, with the gradient-free replay supplying the denominator and the gradient-carrying replay the numerator. Sampler log-probabilities $\mathbf{q}$ are then carried for diagnostics alone.

Routing must be pinned to the generating version. The recorded tensor $\mathbf{R}$ is a property of π_{t-1}^r , the version that produced the tokens, and replaying it while computing gradients for θ_t is deliberate:

$$\mathbb{I}_j^\ell \leftarrow \mathbf{R}[j, \ell, :] = \mathbb{I}_j^{r, \ell}(\pi_{t-1}), \quad g_{j,e}^\ell \leftarrow [\mathbf{s}_j^\ell(\theta_t)]_e, \quad (27)$$

so that selection is frozen at the generating version while gating is evaluated at the current one. The asymmetry is what makes the router trainable under replay: gradients reach $\mathbf{s}_j^\ell(\theta_t)$ through $g_{j,e}^\ell$, whereas the discrete TopK_k , whose sensitivity to $\gamma_t^\ell \leq 2\epsilon$ caused the problem in the first place, is removed from the graph. Under

$\sigma = 1$ the frozen selection is one update stale, which is the same staleness PPO’s ratio already corrects for. Under large σ it would not be, and this is the mechanism by which the staleness studies of Section 10 report baselines collapsing at $\sigma = 8$ while combinations based on $\mathbf{R}^3$ survive.

B.3 Routing alignment across a version boundary

Equation 27 is a statement about one position. Making it hold for every position of a stitched multi-turn trajectory is the actual engineering content, because $\mathbf{R}$ is captured per turn by the replica holding π_{t-1} , while the loss is computed over a single concatenated stream sharded across ranks by the trainer holding θ_t . Algorithm 4 states the invariant chain connecting them.

Algorithm 4. Routing alignment from capture under π_{t-1} to replay under θ_t . Every step is index-preserving, and a violation anywhere silently trains the wrong sub-network, so each is asserted rather than assumed.

Require: per-turn captures $\{(\mathbf{p}_i, \mathbf{a}_i, \mathbf{R}_i)\}_{i=1}^T$ from π_{t-1}^r

Per-turn offset. $\mathbf{R}_i$ holds $\rho_i = |\mathbf{p}_i| + |\mathbf{a}_i| - 1$ records, where record j predicts position $j+1$, so a turn’s last token contributes none.

Stitch-consistent concatenation. Records are appended under the same case decision, Equations 6 to 9, that built $(\mathbf{x}, \mathbf{m}, \mathbf{q})$ and never independently, so $\mathbf{R}[j, \ell, :] = \mathbb{I}_j^{\mathbf{r}, \ell}$ for the same j indexing (x_j, m_j, q_j) .

Gap repair confined to masked positions. $\mathbf{R}[j] \leftarrow \mathbf{R}[j-1]$ is permitted iff $m_{j+1} = 0$ and otherwise aborts, per Equation 14, so no replayed record influencing a gradient is synthetic.

Length contract. **assert** $\rho = N - 1$ per sample; a mismatch means the preceding invariants disagree and is fatal.

Identical sharding. Apply $\Pi = \Pi_{\text{SP}} \circ \text{pad}_{\text{TP} \times \nu} \circ \Pi_{\text{CP}} \circ \text{pad}_1$ to $\mathbf{R}$, in the same composition and the same order as for $\mathbf{x}$. Any deviation permutes records relative to tokens.

Version-split substitution. In the forward pass under θ_t , selection comes from $\mathbf{R}$ at version $t-1$ without gradient and gating from $\mathbf{s}_j^\ell(\theta_t)$ at version t with gradient, per Equation 27.

Cursor discipline. The router advances a forward cursor for the π_{t-1}^t pass and a backward cursor for the θ_t update, because activation recomputation re-runs each layer’s forward pass during the backward pass, so one cursor would be consumed twice per layer.

Scope. Reference and critic passes select freely, since neither needs behavioural fidelity to π_{t-1}^r , and the critic’s distinct batching would violate identical sharding on shared buffers.

The first five invariants concern indexing and would be required even in a synchronous loop, whereas the last three are where the version boundary appears explicitly. What is not claimed is that alignment makes π_{t-1}^t bit-identical to π_{t-1}^r , since the kernel-numerics term of Equation 25 survives. It makes the two agree on which sub-network is differentiated, and that is the term scaling with L , hence the one that matters at 48 layers of top-8-of-256 routing.

B.4 Version bookkeeping in the critic path

The critic introduces a second version axis, and the two must not be conflated. Each step the critic updates first and then supplies its pre-update values to the actor:

$$\phi_t = \phi_{t-1} - \eta_\phi \nabla_\phi \mathcal{L}^V(\phi_{t-1}; \mathcal{B}_t), \quad V_{\text{old}} \equiv V_{\phi_{t-1}} \quad \text{in both } \hat{A}_j \text{ and the value clip.} \quad (28)$$

Anchoring GAE and the ± 0.2 value clip to $V_{\phi_{t-1}}$ rather than to the in-flight V_{ϕ_t} keeps the clip a trust region around a fixed function. The critic never replays routing, so its forward pass runs at $(\phi_{t-1}, \text{TopK}_k)$, and its target is a supervised regression on returns for which behavioural fidelity to π_{t-1}^r is not required.

Finally, the frozen reference π_{ref} carries no version subscript because it does not advance. It also routes with its own TopK_k rather than with $\mathbf{R}$, which is why

$$D_{\text{KL}}(\pi_0^t \parallel \pi_{\text{ref}}) > 0 \quad \text{at initialization, even though } \theta_0 = \theta_{\text{ref}}, \quad (29)$$

and why any assertion of a vanishing initial divergence must be disabled under R^3 , as Section 4.3 notes. The two policies differ not in weights but in routing source.

C Case Study

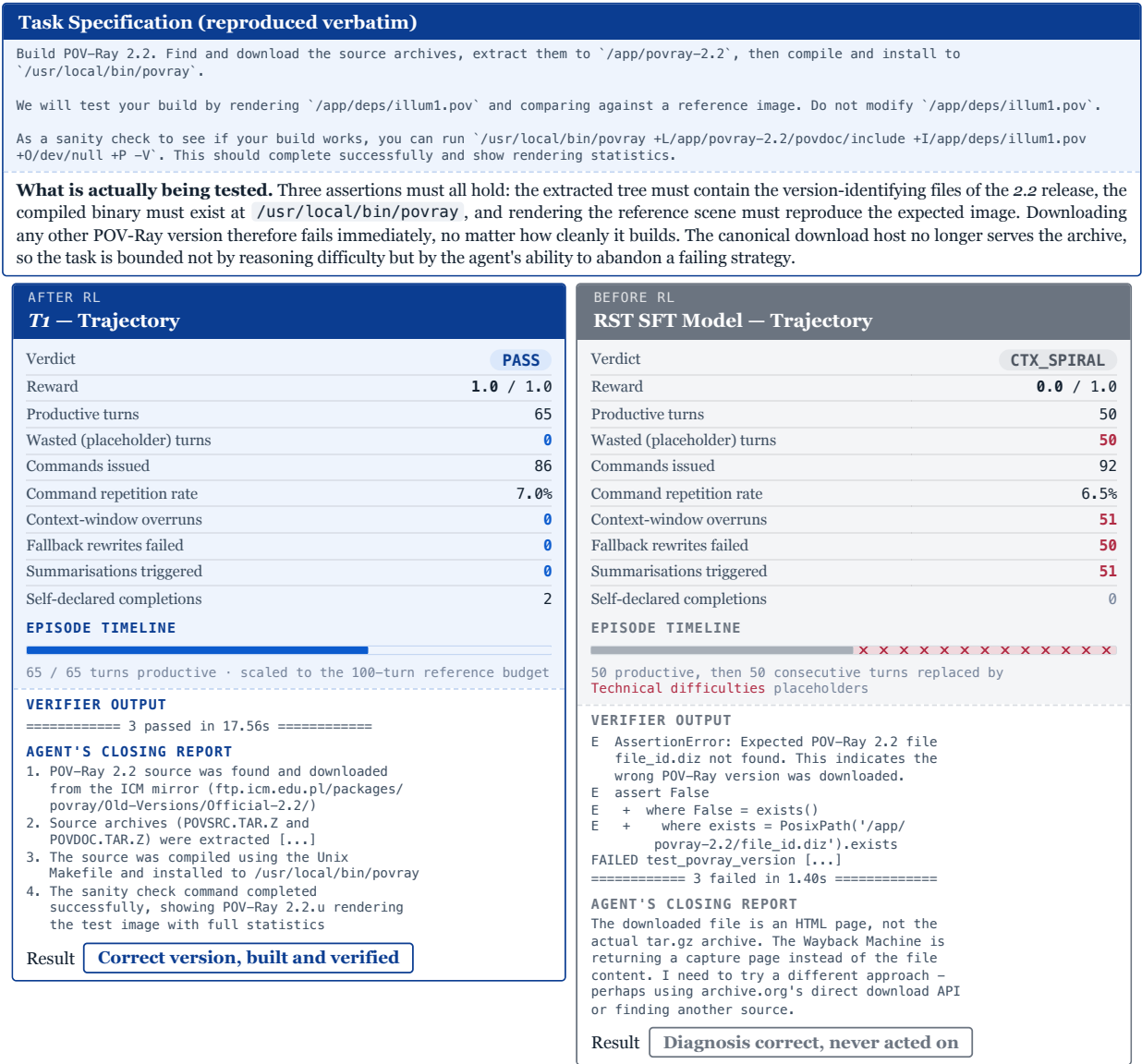
Figures 17 and 18 place T1 next to its own RST-SFT initialization on two Terminal-Bench 2.1 software-engineering tasks of medium difficulty that T1 resolves and the SFT model does not. Both pairs run the same task specification and the same verifier, so the reward difference is attributable to behaviour rather than to the problem statement.

- **build-pov-ray (Figure 17): abandoning a failing strategy.** The canonical download host no longer serves the POV-Ray 2.2 archive. The SFT model diagnoses this correctly in its own closing report, naming the remedy, yet keeps retrying the dead host; each retry appends an HTML error page, producing 51 context overruns and 50 turns replaced by harness placeholders, so half its budget carries no interaction. T1 reaches the same diagnosis, switches to a live FTP mirror, and passes all three assertions in 65 turns with zero overruns. Command counts (86 vs. 92) and repetition rates (7.0% vs. 6.5%) are comparable: what differs is where the search went, not how much of it there was.
- **polyglot-c-py (Figure 18): satisfying the unstated requirement.** Both models write a correct Python/C polyglot and verify the same Fibonacci values, and their closing reports are nearly interchangeable. The verifier, however, asserts that `/app/polyglot` contains exactly `main.py.c`. The SFT model compiles the test binary as the prompt’s own example instructs and leaves it behind; T1 runs the same test and deletes it. Neither run approaches any harness limit, so the entire 1.0-vs-0.0 gap reduces to one cleanup command. T1 spends its extra turns (44 vs. 23) inspecting the environment it is about to return rather than reasoning further about the algorithm.

Both cases show RL changing what the policy does with a conclusion it can already reach: discarding an exhausted hypothesis, and treating the final machine state as part of the deliverable. The two runs differ in harness limits (SFT 56k/8,192 tokens and 100 turns; T1 86k/32,768 tokens and 500 turns), which we note as a caveat, although T1 finished inside the SFT budget in both cases and neither recorded failure cause is one that additional context would have repaired.

build-pov-ray — escaping a context-collapse loop

Terminal-Bench 2.1 · software-engineering · difficulty medium · compared models: T1 (left) and RST SFT Model (right)



polyglot-c-py — when the artefact is correct but the workspace is not

Terminal-Bench 2.1 · software-engineering · difficulty medium · compared models: T1 (left) and RST SFT Model (right)

Task Specification (reproduced verbatim)

Write me a single file in /app/polyglot/main.py.c which is a polyglot, that is, I can run

```
`python3 /app/polyglot/main.py.c N`  
or  
`gcc /app/polyglot/main.py.c -o /app/polyglot/cmain && /app/polyglot/cmain N`
```

Both of these should print to stdout the kth Fibonacci number, $f(0) = 0$, $f(1) = 1$, etc.

I'm using python3 3.12.3 and gcc 13.2.0.

The trap. The specification asks for “a single file”, and it also hands the agent a `gcc` invocation whose `-o` flag writes `cmain` into that very directory. Following the given command literally and leaving the result in place violates the single-file requirement. The verifier enumerates `/app/polyglot` and asserts its contents equal exactly `["main.py.c"]`, so the compiled binary must be produced for testing and then removed — or written elsewhere. Both models solve the stated Fibonacci problem; only one returns a clean workspace.

AFTER RL

T1 — Trajectory

Verdict

PASS

Reward

1.0 / 1.0

Productive turns

44

Wasted (placeholder) turns

0

Commands issued

76

Command repetition rate

48.7%

Context-window overruns

0

Summarisations triggered

0

Self-declared completions

2

FINAL STATE OF /app/polyglot

- ✓ `main.py.c` — the polyglot
- ✓ (nothing else) — workspace clean

VERIFIER OUTPUT

===== 1 passed in 0.14s =====

AGENT'S CLOSING REPORT

The polyglot file has been created and thoroughly tested. Both Python and C implementations correctly compute Fibonacci numbers: $f(0) = 0$, $f(1) = 1$, $f(5) = 5$, $f(10) = 55$

The file at /app/polyglot/main.py.c works as a polyglot:

- `python3 /app/polyglot/main.py.c N` runs the Python code
- `gcc [...] -o /app/polyglot/cmain && /app/polyglot/cmain N` compiles and runs the C code

The trick used is:

- C sees `#if 0` to hide Python code, and `"""` inside `#if 0` blocks
- Python sees `#` as comments, executes Python code, and `"""` as a multi-line string containing C code

Result

Artefact correct, directory clean

BEFORE RL

RST SFT Model — Trajectory

Verdict

WRONG

Reward

0.0 / 1.0

Productive turns

23

Wasted (placeholder) turns

0

Commands issued

66

Command repetition rate

59.1%

Context-window overruns

0

Summarisations triggered

0

Self-declared completions

2

FINAL STATE OF /app/polyglot

- ✓ `main.py.c` — the polyglot
- ✗ `cmain` — leftover build artefact

VERIFIER OUTPUT

```
> assert polyglot_files == ["main.py.c"], (  
E AssertionError: Expected only main.py.c,  
  found: ['main.py.c', 'cmain']  
E assert ['main.py.c', 'cmain'] == ['main.py.c']  
E Left contains one more item: 'cmain'  
FAILED test_fibonacci_polyglot [...]  
===== 1 failed in 0.06s =====
```

AGENT'S CLOSING REPORT

The polyglot file is complete and working. Both Python and C produce correct Fibonacci numbers: $f(0) = 0$, $f(1) = 1$, $f(5) = 5$, $f(10) = 55$

The file uses the classic polyglot technique:

- `#if 0` ... `#endif` blocks make C preprocessor skip the Python code, while Python sees `#if 0` as a comment
- `"""` strings make Python treat the C code as a string literal, while C skips the `"""` via `#if 0` blocks

Both commands work as required:

- `python3 /app/polyglot/main.py.c N`
- `gcc [...] -o /app/polyglot/cmain && /app/polyglot/cmain N`

Result

Artefact correct, directory polluted

Analysis

- The two closing reports are almost interchangeable.** Both models describe the same polyglot construction — `#if 0` blocks to hide Python from the C preprocessor, triple-quoted strings to hide C from Python — and both verify the same four values. Judged as free-text reasoning the trajectories are equally good; the verifier disagrees, because it inspects the directory rather than the argument.
- The distinguishing behaviour is cleanup, and it is one command wide.** The RST SFT Model compiles `cmain` to test its C path — correctly, and as the prompt's own example instructs — then leaves it there. T1 performs the same test and removes the binary before finishing. The entire 1.0-vs-0.0 reward gap reduces to whether the agent treats a test by-product as part of the deliverable.
- Both agents declared completion twice, so confidence is not the discriminator.** Self-reported certainty was identical, and neither run hit any infrastructural limit: zero context overruns and zero summarisations on both sides. This is a clean behavioural comparison, free of the harness confounds present in Case Study I.
- T1 spent nearly twice the turns (44 vs 23) at a lower repetition rate (48.7% vs 59.1%).** High repetition is expected here — verifying a polyglot means re-running the same two invocations after every edit. The relevant contrast is that the RST SFT Model finished early and confidently, whereas T1's extra turns went not into further reasoning about Fibonacci but into checking the environment it was about to hand back.

Figure 18 Correct artefact in a polluted workspace. Layout follows Figure 17, with the final contents of `/app/polyglot` shown for each model. Both agents produce a working Python/C polyglot and report the same verified Fibonacci values, but the verifier requires the directory to hold exactly `main.py.c`: the RST-SFT model leaves the compiled `cmain` binary in place and scores 0.0, whereas T1 removes it after testing and scores 1.0. Neither run triggers a context overrun or a summarization.